



Arquitectura de Software

18 de Julio de 2026

Tabla de contenido

1. Introducción	7
1.1 Contexto del proyecto	7
1.2 Descripción general de la solución	8
1.3 Objetivo del documento	9
2. Propósito	9
3. Alcance	10
3.1 Qué cubre	10
3.2 Qué NO cubre	11
4. Drivers Arquitectónicos	11
4.1 Tabla de drivers	12
4.2 Desarrollo de los drivers principales	14
5. Restricciones técnicas	14
6. Atributos de calidad prioritarios	16
6.1 Matriz de atributos	16
6.1 Justificación de la priorización	18
7. Principios arquitectónicos	18
8. Estilo de arquitectura	20
8.1 Microservicios	20
8.2 Arquitectura orientada a eventos (EDA)	21
8.3 API Gateway	22
8.4 Base de datos por servicio	23
8.5 Estilos internos	24
9. Diagrama C4 de contexto	24
Explicación del diagrama	24
10. Diagrama C4 de contenedores	25
Explicación del diagrama	25
11. Diagramas de componentes	26
11.1 API Gateway	26
11.2 identity-service	27
11.3 products-service	28

11.4 orders-service.....	28
11.5 financial-service.....	29
11.6 fulfillment-service.....	29
11.7 notifications-service.....	30
11.8 reporting-service.....	31
12. Descripción de cada microservicio.....	31
12.1 identity-service	31
12.2 products-service.....	32
12.3 orders-service.....	33
12.4 financial-service.....	34
12.5 fulfillment-service	35
12.6 notifications-service.....	35
12.7 reporting-service	36
12.8 API Gateway (contenedor de soporte).....	37
13. Comunicación síncrona mediante REST	37
13.1 Modelo general.....	37
13.2 Matriz de llamadas REST internas (servicio→servicio)	38
13.3 API pública por el gateway (resumen de superficie).....	39
14. Comunicación asíncrona mediante eventos	40
14.1 Infraestructura de mensajería	40
14.2 El sobre estándar	40
14.3 Publicación confiable: Transactional Outbox	41
14.4 Consumo confiable: idempotencia y reentrega	41
14.5 Flujo maestro de la saga de venta (coreografía)	42
14.6 Flujo de devolución	43
15. Comunicación en tiempo real mediante WebSockets.....	43
15.1 Topología.....	43
15.2 Autenticación del canal	43
15.3 Contrato de eventos del chat (orders).....	44
15.4 Contrato del canal de notificaciones	44
15.5 Consideraciones de escala.....	45

16. Diagrama de interacción entre servicios	45
Explicación del diagrama	45
17. Catálogo de eventos	46
17.1 Dominio identity (productor: identity-service)	46
17.2 Dominio order (productor: orders-service)	47
17.3 Dominio products (productor: products-service)	48
17.4 Dominio financiero (productor: financial-service)	49
17.5 Dominio fulfillment (productor: fulfillment-service)	50
17.6 Canal de comandos	51
18. Vista de datos	51
18.1 Estrategia de persistencia	51
18.2 Dueños de datos por concepto de negocio	52
19. Diagramas de bases de datos	53
19.1 identity_db	53
19.2 products_db	53
19.3 orders_db	54
19.4 financial_db	54
19.5 fulfillment_db	55
19.6 notifications_db	55
19.7 Nota sobre reporting	56
20. Diagramas de secuencia relevantes	56
20.1 Registro e inicio de sesión	56
20.2 Creación de pedido (carrito → checkout)	56
20.3 Validación y reserva de inventario	57
20.4 Pago aprobado	57
20.5 Pago rechazado	58
20.6 Generación del QR	58
20.7 Confirmación de entrega	58
20.8 Cancelación y reembolso	59
20.9 Notificación de cambio de estado	59
20.10 Chat entre comprador y vendedor	59

21. Arquitectura general de seguridad	60
21.1 Autenticación	60
21.2 Autorización	61
21.3 Anti-spoofing y saneamiento	61
21.4 Gestión de secretos y cifrado	62
21.5 CORS, filtros e interceptores	62
21.6 Seguridad entre servicios	62
21.7 Buenas prácticas adicionales verificadas	63
22. Arquitectura general de observabilidad	63
22.1 Telemetría centralizada	63
22.2 Logs	63
22.3 Métricas y KPIs	64
22.4 Health checks	64
22.5 Monitoreo activo y alertas	64
22.6 Tracing distribuido	65
23. Diagrama de despliegue	65
Explicación del diagrama	65
24. Ambiente de desarrollo y producción	66
24.1 Ambientes identificados	66
24.2 Diferencias clave entre ambientes	66
24.3 Pipeline de entrega (backend)	67
25. Stack tecnológico	67
25.1 Frontend	67
25.2 Backend	68
25.3 Mensajería	69
25.4 Persistencia	69
25.5 Cloud e infraestructura	69
25.6 CI/CD	69
25.7 Servicios externos (SaaS)	70
25.8 Herramientas de desarrollo	70
26. Resumen de decisiones arquitectónicas	71

27. Riesgos técnicos	73
28. Deuda técnica	75
29. Glosario técnico	78

1. Introducción

1.1 Contexto del proyecto

ECIEXPRESS es una plataforma de pedidos anticipados con retiro en punto de venta para el campus de la Escuela Colombiana de Ingeniería Julio Garavito. El problema de negocio que resuelve es la congestión y las filas en las cafeterías, papelerías y restaurantes del campus durante las franjas de alta demanda: el estudiante o empleado ordena y paga desde la web antes de llegar, la tienda prepara el pedido, y el comprador lo retira presentando un código QR de retiro (o un código corto tecleable) sin hacer fila.

Los flujos de negocio centrales soportados por el sistema, son:

- **Identidad y administración:** registro/login federado con Firebase Auth, perfiles locales, RBAC (roles BUYER, VENDOR, ADMIN, ANALYST), administración de tiendas (tipos CAFETERIA, PAPELERIA, RESTAURANTE), horarios semanales, cierres temporales programados, staff por tienda, auditoría y tickets PQRS.
- **Catálogo y stock:** productos por tienda con categorías jerárquicas, promociones con precio efectivo, movimientos de inventario, reserva de stock por pedido, alertas de stock bajo y generación automática de modelos 3D de los productos a partir de fotografías (pipeline de IA con Tencent Hunyuan3D).
- **Pedidos y comunicación:** carrito (orden en estado DRAFT), cotización autoritativa por el servicio de productos, checkout con recargo de hora pico, máquina de estados del pedido con 13 estados, devoluciones con evidencia fotográfica y aprobación administrativa, calificaciones, y chat comprador–vendedor en tiempo real por WebSocket.
- **Dinero:** billetera virtual por usuario (saldo en centavos de COP), recargas vía la pasarela Wompi (tarjeta tokenizada, Nequi, PSE, Bancolombia), retención tipo escrow del cobro (HELD), liberación al negocio al confirmarse la entrega (RELEASED), comisión de plataforma, recargo de hora pico, reembolsos con políticas diferenciadas y recibos PDF.
- **Entrega:** generación del código de retiro al quedar el pedido listo, validación del QR por el vendedor, confirmación de entrega (QR o manual), registro de entregas fallidas y expiración programada de códigos.
- **Notificaciones:** bandeja in-app persistente, correo (Gmail API), WhatsApp (Meta Cloud API), SMS (Twilio) y tiempo real (Socket.IO), con plantillas HTML por evento y preferencias por usuario.

- **Reporting y monitoreo:** KPIs técnicos (volumen, tasa de error, p95, flujo de eventos) sobre Azure Application Insights, consultas de negocio de solo lectura y estado de despliegues, para los roles ADMIN/ANALYST.

1.2 Descripción general de la solución

La solución está construida como un sistema distribuido de 7 microservicios NestJS más un API Gateway propio (también NestJS) y una aplicación web React/Vite. Los servicios se comunican de forma síncrona por REST (para consultas de autoridad en línea) y de forma asíncrona por eventos de dominio sobre un topic compartido de Azure Service Bus (eciexpress_events), con patrón Transactional Outbox en los servicios productores y deduplicación por clave de idempotencia en los consumidores. Cada servicio posee su propia base de datos PostgreSQL (aprovisionada en Neon). El despliegue productivo se realiza en Azure Container Apps mediante pipelines de GitHub Actions con autenticación OIDC, y el frontend en Azure Static Web Apps.

Componente	Rol
gateway	Única puerta de entrada; autenticación Firebase, enriquecimiento de identidad, rate limiting, proxy HTTP y WebSocket
Identity-service	Fuente de verdad de identidad, RBAC, tiendas, sesiones, auditoría, PQRS
products-service	Catálogo, categorías, promociones, inventario, reservas de stock, activos 3D
orders-service	Dueño del ciclo de vida del pedido, carrito, devoluciones, chat
financial-service	Billetera, recargas Wompi, retención/liberación de pagos, comisiones, recibos
fulfillment-service	Códigos de retiro (QR), validación, confirmación de entrega, expiración
notifications-service	Notificaciones multicanal y bandeja in-app

reporting-service	KPIs, monitoreo, consultas de negocio de solo lectura, estado de infraestructura
frontend/ecixpress	SPA React para compradores, vendedores, administradores y analistas
hunyuan3d-ap	API Python de generación de modelos 3D (Tencent Hunyuan3D) sobre GPU

1.3 Objetivo del documento

Este documento consolida la arquitectura real implementada del sistema ECIEXPRESS: sus drivers y restricciones, el estilo arquitectónico y sus justificaciones, las vistas C4 (contexto, contenedores, componentes), los contratos de comunicación síncrona/asíncrona/tiempo real, el catálogo completo de eventos, la vista de datos, la seguridad, la observabilidad, el despliegue, las decisiones arquitectónicas tomadas (ADRs), los riesgos y la deuda técnica observada. Sirve como referencia única para diseño, mantenimiento, incorporación de nuevos integrantes y auditoría.

2. Propósito

Este documento existe por tres razones concretas:

- 1. Servir de contrato entre equipos.** Cada microservicio es mantenido por un equipo distinto (los repositorios tienen .git independientes y responsables propios). Las fronteras de responsabilidad ("Order es el único dueño del estado del pedido", "Fulfillment no mueve dinero"), los sobres de eventos y las convenciones (montos en centavos COP, JSON plano, idempotencyKey) son acuerdos inter-equipo que aquí quedan formalizados.
- 2. Soportar decisiones futuras.** Las secciones de ADRs, riesgos y deuda técnica dan trazabilidad de por qué el sistema es como es (p. ej. migración de RabbitMQ/CloudAMQP a Azure Service Bus) y qué debe atenderse a continuación.
- 3. Facilitar la operación.** Seguridad (rutas públicas/bloqueadas del gateway, manejo del webhook de Wompi), observabilidad (Application Insights, logging estructurado con correlación) y despliegue (Container Apps, revisiones, ACR) quedan descritos para quien opere la plataforma.

3. Alcance

3.1 Qué cubre

Área	Cobertura
Arquitectura de backend	Los 7 microservicios NestJS + API Gateway: módulos, componentes, endpoints, guards, persistencia, mensajería
Comunicación	REST síncrono (externo e interno), eventos asíncronos (Azure Service Bus), WebSockets (chat y notificaciones)
Datos	Estrategia database-per-service, esquemas por servicio (Prisma/TypeORM/migraciones), almacenamiento de objetos (Azure Blob Storage), cachés (Redis)
Seguridad	Autenticación Firebase, sesión única, RBAC y permisos, inyección de headers de identidad, rutas públicas/bloqueadas, webhook firmado de Wompi, secretos
Observabilidad	Application Insights, logging estructurado, correlación, health checks, KPIs del reporting-service
Despliegue	Azure Container Apps, Azure Container Registry, Azure Static Web Apps, GitHub Actions (OIDC), stack local de integración con Docker Compose y emulador de Service Bus
Frontend	Arquitectura de alto nivel de la SPA (React/Vite), integración con gateway, Firebase, Wompi y WebSockets
Servicio de IA 3D	API Hunyuan3D (Python) y su integración con products-service

3.2 Qué NO cubre

Área excluida	Dónde se trata / Motivo
Especificación funcional de requerimientos	Documento 02 de la serie
Diseño visual, sistema de diseño y UX	Documento 06 de la serie
Estrategia de pruebas y calidad en detalle	Documento 04 de la serie (aquí solo se referencia la existencia de pruebas unitarias/e2e y su rol en el pipeline)
Gestión del proyecto, cronograma, equipo	Documento 10 de la serie
Negocio, marketing y adopción	Documento 09 de la serie
Detalle interno del modelo de IA Hunyuan3D (arquitectura del modelo, pesos, entrenamiento)	Se documenta solo como servicio externo consumido; detalle en hunyuan3d-ap/README.md
Definición de la infraestructura como código (Terraform)	Los workflows y comentarios del código referencian outputs y reglas creadas "vía Terraform", pero el código Terraform no está en este repositorio. Información pendiente de confirmar con el equipo de desarrollo (ubicación del repo de laC)
Aplicación móvil nativa	No existe en el repositorio; el orden soporta source: 'mobile' como valor de enum, pero el único cliente implementado es la web

4. Drivers Arquitectónicos

Los drivers se identificaron a partir de decisiones observables en el código y la configuración. Para cada uno se indica la evidencia que demuestra que efectivamente condicionó la arquitectura.

4.1 Tabla de drivers

#	Driver	Por qué existe	Evidencia en el repositorio
D-01	Modularidad y autonomía de equipos	El proyecto se desarrolla por equipos paralelos (uno por dominio) que necesitan desplegar sin coordinarse	Un repositorio Git independiente por servicio (cada carpeta tiene su .git); un pipeline azure-deploy.yml por servicio; catálogos de eventos escritos como contratos entre equipos (orders-service/docs/CATALOGO-DE-EVENTOS.md incluye "Checklist para quien publique hacia orders-service")
D-02	Consistencia del negocio ante fallos parciales	Un pedido toca 5 servicios (orders, products, financial, fulfillment, notifications); ninguna operación distribuida puede dejar dinero o stock en estado inconsistente	Patrón Transactional Outbox en identity, orders y fulfillment (outbox_events + workers con reintentos); deduplicación por idempotencyKey en todos los consumidores (processed_events); advisory locks de PostgreSQL en fulfillment (pg_advisory_xact_lock) y unicidad por order_id en financial
D-03	Tiempo real	El chat comprador-vendedor y las notificaciones in-app requieren push inmediato; el vendedor debe enterarse del pedido entrante al instante	Socket.IO en orders (/orders/socket.io, namespace /communication) y notifications (/notifications/socket.io); proxy de WebSocket en el gateway (ws-proxy.service.ts)
D-04	Seguridad perimetral centralizada	Solo el gateway debe ser accesible desde internet; los servicios confían en headers inyectados y no repiten validación de tokens	docker-compose.yml: "Solo el gateway se expone al host (3000)... aproximando el aislamiento de red que tendrán en Azure Container Apps"; header-injector.ts elimina headers de identidad del cliente antes de inyectar los reales; rutas /identity/internal/* bloqueadas con 404
D-05	Escalabilidad elástica y costo	Plataforma universitaria con picos de demanda muy marcados	Azure Container Apps (escala por revisiones), rate limiting distribuido con Redis para múltiples réplicas del gateway (throttle.storage.ts), wakeup.service.ts diseñado para despertar servicios con cold start

		(horas de almuerzo) y valles largos; el costo debe seguir a la demanda	
D-06	Trazabilidad y auditabilidad	Operaciones sensibles (dinero, roles, cierres de tienda) exigen rastro completo	audit_logs append-only en identity y fulfillment; order_status_history con actor y razón por transición; x-correlation-id generado en el gateway y propagado en HTTP y en el sobre de todos los eventos
D-07	Precio visto = precio cobrado	Regla de negocio explícita: el total que el comprador ve en el checkout debe ser exactamente lo que se debita	orders.service.ts#checkout: "financiamiento EXACTAMENTE este total sin recalcular el pico (el precio que vio el comprador == el precio cobrado)"; el desglose (orderAmount, peakFeeAmount, totalAmount) viaja en order.order.created
D-08	Evitar sobreventa de inventario	Dos compradores no pueden llevarse la última unidad	Eventos product.inventory.reservation_confirmed/rejected; en orders, el flag stockReserved gatea la confirmación de pedidos digitales y los pedidos en efectivo esperan la reserva antes de CONFIRMED ("Option C" en order.models.ts y orders.service.ts)
D-09	Operabilidad con equipo pequeño	No hay equipo de plataforma dedicado; la observabilidad y el monitoreo deben ser autoservicio	Application Insights en los 8 servicios (applicationinsights en todos los package.json); reporting-service expone KPIs, salud de servicios, estado de deploys y métricas del bus para el rol ADMIN/ANALYST
D-10	Experiencia de producto diferenciada	El catálogo compete en experiencia: vista 3D de productos y mapa del campus	Pipeline 3D en products-service (campos model3dUrl, modelGenerationStatus/Progress/Error, endpoints /:id/model3d, /:id/retry-3d, ai-health); frontend usa @google/model-viewer, maplibre-gl y qr-scanner

4.2 Desarrollo de los drivers principales

D-02 — Consistencia distribuida. Es el driver que más forma le dio al sistema. La venta involucra tres invariantes que viven en servicios distintos: (a) el saldo de la billetera (financiamiento), (b) el stock físico y reservado (products), y (c) el estado del pedido (orders). La arquitectura resuelve la coordinación con una saga coreografiada por eventos: `order.order.created` dispara en paralelo la retención del cobro y la reserva de stock; el pedido solo pasa a `CONFIRMED` cuando confluyen `financial.payment.processed` y `product.inventory.reservation_confirmed` (pedidos digitales), o cuando llega la reserva (pedidos en efectivo). Los mecanismos de soporte (outbox, idempotencia, advisory locks, `wasSold` para distinguir liberar-reserva vs. restituir-stock) existen precisamente para que esa coreografía sobreviva a reintentos, mensajes duplicados y caídas del broker.

D-04 — Perímetro único. Todo el modelo de confianza descansa en que el único proceso alcanzable desde internet es el gateway: valida el token de Firebase, resuelve la identidad local (con caché Redis de 60 s), rechaza usuarios no `ACTIVE`, elimina cualquier header de identidad enviado por el cliente e inyecta los verificados (`x-user-id`, `x-user-role`, `x-user-store`). Los servicios internos operan en “modo header” (p. ej. orders con `AUTH_DISABLED=true` detrás del gateway) — una decisión que solo es segura si el aislamiento de red del perímetro se mantiene (ver Riesgo R-02).

D-05 — Elasticidad. La plataforma tiene un patrón de carga fuertemente estacional (picos al mediodía). Container Apps permite escalar réplicas por servicio de forma independiente, y las piezas con estado compartido (rate limiting, caché de enriquecimiento) se apoyan en Redis justamente para que el gateway pueda tener N réplicas sin perder coherencia de conteo.

5. Restricciones técnicas

Las restricciones son condiciones impuestas (por plataforma, negocio o contexto académico) que el diseño debió acatar.

#	Restricción	Tipo	Descripción y evidencia
R-01	Stack Node.js/NestJS + TypeScript estricto en todo el backend	Tecnológica	Los 8 servicios usan NestJS 11 sobre Node 20/22 con TypeScript. Evidencia: <code>package.json</code> de cada servicio; workflows usan Node 22 (backend) y Node 20 (frontend)
R-02	Nube Microsoft Azure para cómputo, mensajería,	Infraestructura	Azure Container Apps (workflows <code>azure-deploy.yml</code> , grupo <code>eciexpress-prod-rg</code>), Azure Container Registry, Azure Service Bus (<code>@azure/service-bus</code>), Azure Blob Storage

	almacenamiento y telemetría		(@azure/storage-blob), Azure Application Insights (applicationinsights), Azure Static Web Apps (frontend), Azure Cache for Redis (rediss://...redis.cache.windows.net en gateway/.env.example)
R-03	Autenticación delegada en Firebase Auth	Tecnológica/Seguridad	Las credenciales nunca tocan las bases propias: "Firebase Auth es la única fuente de verdad de autenticación" (identity-service/README.md). firebase-admin en gateway e identity
R-04	PostgreSQL gestionado (Neon) como única familia de base de datos relacional	Infraestructura	Comentario en docker-compose.yml ("Postgres (Neon)", README.md de identity ("PostgreSQL (Neon en producción)"), drivers pg en todos los servicios con BD
R-05	Pasarela de pagos Wompi (sandbox)	Negocio/Legal	wompi.service.ts + VITE_WOMPI_BASE_URL: https://sandbox.wompi.co/v1 en el workflow del frontend. Nota legal en el código: para operar dinero real se requiere regulación SFC — el sistema corre en sandbox
R-06	Moneda única COP con montos enteros en centavos	Negocio	Convención transversal documentada en todos los catálogos de eventos y aplicada en entidades (bigint con transformer en financial; enteros en orders/products)
R-07	GPU NVIDIA para la generación 3D, fuera de Azure	Infraestructura	hunyuan3d-ap/README.md: requiere GPU NVIDIA (recomendado 24 GB VRAM); "El proyecto fue desplegado sobre una instancia AWS EC2... NVIDIA A10G GPU". products-service lo consume vía PRODUCT_AI_GENERATE_URL con timeout de 10 min
R-08	Suscripción Azure con capacidades limitadas	Infraestructura	Los workflows documentan bloqueos de la suscripción: "az acr build / ACR Tasks está bloqueado en esta suscripción (TasksOperationsNotAllowed)" — por eso el build usa Docker en el runner y la limpieza de imágenes se hace por data-plane
R-09	Presupuesto académico / servicios free-tier	Contexto	Neon (Postgres serverless), Wompi sandbox, y el wakeup.service.ts con espera de 90 s para cold starts (herencia del despliegue previo en Render) evidencian operación en capas gratuitas o de bajo costo

R-10	Frontend SPA servido como estático	Tecnológica	Azure Static Web Apps + Vite: toda la configuración del cliente va en variables VITE_* embebidas en el bundle en build time (workflow azure-swa-deploy.yml)
R-11	CORS restringido a orígenes del frontend	Seguridad	CORS_ORIGINS en gateway (compose: http://localhost:5173, http://localhost:4200; prod: URL del SWA)
R-12	Compatibilidad retro entre versiones de eventos	Evolución	financial acepta eventos "legados" sin desglose de hora pico (orderAmount opcional con fallback a totalAmount en order.payloads.ts); el sobre lleva eventVersion

6. Atributos de calidad prioritarios

Priorización derivada de los drivers y de dónde el código invierte esfuerzo real (los atributos con mecanismos implementados pesan más que los declarativos).

6.1 Matriz de atributos

Prioridad	Atributo	Escenario de calidad (estímulo → respuesta)	Mecanismos implementados (evidencia)
1	Confiabilidad / Consistencia	El broker de eventos cae 10 min durante una venta → ningún evento se pierde; al volver, la saga continúa y no hay doubles cobros ni doubles reservas	Outbox transaccional con worker de reintentos y backoff (identity: 2^retryCount, máx. 5 → FAILED; orders: poll 5 s, 5 reintentos); processed_events + idempotencyKey en consumidores; unicidad order_id en order_transactions (código PG 23505 tratado como idempotente); advisory locks en fulfillment
2	Seguridad	Un cliente envía x-user-id falsificado → el gateway lo elimina e inyecta la identidad verificada del token	Ver sección 21: verificación Firebase, enriquecimiento, strip de headers, rutas bloqueadas, sesión única con X-Session-Id, RBAC con permisos granulares, webhook Wompi verificado por checksum SHA-256 sobre body crudo
3	Disponibilidad / Resiliencia	Redis cae → el gateway sigue operando (rate limit en memoria, enriquecimiento sin	Degradación explícita en redis.service.ts ("degradación a null si no disponible") y en financial-http.client.ts ("nunca un checkout bloqueado"); restart: unless-

		caché); financial no responde → la cotización del carrito no falla (recargo 0, el cobro real lo fija financial)	stopped; health checks (/health, terminus en products con live/ready)
4	Escalabilidad	Pico de mediodía multiplica el tráfico ×10 → réplicas adicionales del gateway comparten el conteo de rate limit y el caché en Redis; los servicios escalan de forma independiente	Redis distribuido (GatewayThrottlerStorage con INCR/PEXPIRE), Container Apps por servicio, consumidores del bus con maxDeliveryCount y DLQ
5	Mantenibilidad / Evolucionabilidad	Un equipo cambia su implementación interna → los demás no se enteran (contratos estables de eventos y REST)	Arquitectura hexagonal en orders (puertos EventPublisher, FinancialPort, IdentityPort, ProductsPort con adaptadores HTTP/mock); feature modules en products; ServiceBusService que "expone la misma API que el antiguo RabbitMQService para no tocar al consumer ni al worker" — la migración de broker no tocó lógica de negocio
6	Observabilidad	Un pedido se atasca → con el correlationId se rastrea la cadena HTTP + eventos en Application Insights; el ADMIN ve p95 y errores en el centro de monitoreo	App Insights en los 8 servicios; loggers estructurados (pino / winston / StructuredLogger propio); reporting-service con KQL (overview, latency, events)
7	Interoperabilidad	Terceros llaman al sistema (Wompi) y el sistema llama a terceros (Gmail, Twilio, Meta, Hunyuan3D) sin acoplar el dominio	Webhook público con verificación de firma; canales de notificación como adaptadores intercambiables (channel.interface.ts); IA 3D detrás de una URL configurable
8	Rendimiento	Consultas del catálogo y validación de identidad en el hot	gw:enrich:<uid> en Redis; índices IDX_products_store_active, IDX_products_category;

	path → caché de enriquecimiento (TTL 60 s) evita llamar a identity en cada request; índices dedicados	statement_timeout 8 s y pools acotados en reporting
--	---	---

6.1 Justificación de la priorización

La consistencia encabeza la lista porque es el atributo con más mecanismos redundantes implementados (outbox + idempotencia + locks + políticas de compensación) y porque el dominio maneja dinero: un evento perdido entre `order.order.created` y `financial.payment.processed` dejaría un cobro retenido sin pedido confirmado. La seguridad es segunda porque el perímetro concentra toda la autenticación y una falla ahí compromete a los 7 servicios a la vez. La disponibilidad se diseñó pragmáticamente como degradación elegante (mejor responder sin caché que no responder) más que como alta disponibilidad formal — no hay evidencia de SLAs numéricos definidos (Información pendiente de confirmar con el equipo de desarrollo).

7. Principios arquitectónicos

Principios extraídos de patrones consistentes en el código — no son aspiraciones, sino reglas que el sistema efectivamente cumple:

- 1. Un único dueño por dato y por decisión.** Cada agregado tiene exactamente un servicio dueño: el estado del pedido lo decide `orders` (“Order es el único dueño del estado del pedido: `financial`, `fulfillment` y `products` solo publican; aquí se deciden las transiciones” — `CATALOGO-DE-EVENTOS.md`); el precio autoritativo lo fija `products` (`products.cart.priced`); la comisión y la hora pico las fija `financial`; la identidad y las tiendas, `identity`. Los demás mantienen proyecciones locales de solo lectura.
- 2. Base de datos por servicio;** nunca acceso cruzado de escritura. Cada servicio tiene su BD PostgreSQL propia. La única excepción, explícita y de solo lectura, es `reporting-service`, que consulta las BDs de `orders` y `financial` con un catálogo `allowlisted` de consultas parametrizadas (`business-db.service.ts`: “NUNCA se interpola entrada del usuario en el SQL”).
- 3. Eventos primero para hechos;** REST para preguntas. Los hechos de negocio (se creó un pedido, se aprobó un pago) viajan como eventos inmutables por el

bus; las consultas que necesitan respuesta inmediata y autoritativa (validar token, cotizar comisión, disponibilidad de tienda) van por REST interno.

4. **Publicación confiable o nada (Outbox).** “No publicamos directo... usamos un outbox para garantizar que ningún evento se pierda aunque el broker esté caído”. El evento se persiste en la misma transacción que el cambio de negocio y un worker lo publica con reintentos.
5. **Todo consumidor es idempotente.** Los eventos llegan al-menos-una-vez; reprocesar nunca produce efectos dobles (deduplicación por idempotencyKey, restricciones únicas, advisory locks).
6. **Notificación de cambio $\neq$ transferencia de estado.** Cuando el payload no puede garantizar el snapshot completo, el evento solo dispara una reconciliación pull contra el endpoint interno del dueño (products ante identity.store.*: “el evento se trata como notificación de cambio y no como fuente de datos”).
7. **Confianza en el perímetro, desconfianza en el cliente.** El gateway verifica y enriquece; los servicios internos confían en headers inyectados; todo header de identidad proveniente del cliente se elimina antes de proxyear.
8. **Contratos explícitos y versionados.** Sobre estándar de 5 metadatos (eventVersion, source, correlationId, occurredAt, idempotencyKey), JSON plano sin subnodo payload, routing keys servicio.entidad.accion, montos en centavos COP. Los contratos viven en archivos dedicados (event-contracts.ts, contracts.ts, event-patterns.ts).
9. **Puertos y adaptadores donde hay volatilidad.** orders define puertos de aplicación e inyecta adaptadores HTTP o mock (products.mock.client.ts); los canales de notificación implementan una interfaz común; el ServiceBusService reemplazó al RabbitMQService conservando la firma.
10. **Degradación elegante sobre acoplamiento a dependencias.** Redis caído no tumba el gateway; financiero caído no bloquea el checkout; plantilla HTML inexistente cae a texto plano; canal sin credenciales opera en sandbox.
11. **Auditabilidad por defecto en operaciones sensibles.** Logs de auditoría append-only, historial de estados con actor/razón, y correlación de extremo a extremo.
12. **Automatización de despliegue reproducible.** Build $\rightarrow$ test $\rightarrow$ imagen $\rightarrow$ ACR $\rightarrow$ revisión nueva de Container App, con OIDC (sin secretos de larga vida) y --revision-suffix para forzar revisiones trazables por commit.

8. Estilo de arquitectura

El sistema combina cuatro estilos complementarios: microservicios, arquitectura orientada a eventos (EDA), API Gateway y base de datos por servicio. Internamente, los servicios aplican arquitectura hexagonal/por capas (más marcada en orders y fulfillment) y el frontend es una SPA desacoplada.

8.1 Microservicios

Qué se implementó. Siete servicios de dominio + gateway, cada uno con: repositorio propio, pipeline propio, Dockerfile propio, base de datos propia, ciclo de release independiente y contrato público (REST + eventos). La descomposición sigue los subdominios del negocio (identidad, catálogo, pedidos, dinero, entrega, notificación, analítica) — alineación con Domain-Driven Design a nivel de bounded contexts.

Ventajas (en este proyecto):

Ventaja	Materialización concreta
Despliegue independiente	8 workflows azure-deploy.yml separados; un cambio en fulfillment no redespiega orders
Equipos paralelos	Repos Git independientes con responsables por servicio; contratos formales entre equipos
Aislamiento de fallos	La caída de notifications no impide vender; la de financial degrada solo la cotización del recargo
Escala selectiva	El gateway y orders (chat WS) pueden escalar sin escalar reporting
Heterogeneidad controlada	identity usa Prisma+Zod+pino; products usa TypeORM+class-validator+winston — cada equipo eligió dentro del mismo stack NestJS

Desventajas (y cómo se mitigan aquí):

Desventaja	Cómo se manifiesta / mitigación
Complejidad operacional	9 procesos + Redis + bus para levantar el sistema completo; mitigado con docker-compose.yml de integración local y emulador de Service Bus
Consistencia eventual	El pedido pasa por estados intermedios visibles (PENDING_PAYMENT, espera de reserva); mitigado con saga + outbox + idempotencia y con UX que muestra estados
Duplicación de datos	Proyecciones locales de tiendas en products/financial/fulfillment/notifications y de carrito en products; mitigado con reconciliación pull y eventos de sincronización
Testing de extremo a extremo costoso	Stack local completo dedicado a "PRUEBAS DE INTEGRACIÓN" (compose) y scripts de publicación de eventos de prueba
Latencia inter-servicio	Hot path optimizado: caché de enriquecimiento en el gateway y proyecciones locales para evitar llamadas síncronas en cadena

Por qué se eligió. El proyecto es un dominio naturalmente particionado (cada subdominio tiene invariantes y ritmos de cambio distintos), desarrollado por varios equipos en paralelo bajo evaluación por dominio — la autonomía de repositorio/pipeline/BD era prácticamente un requisito organizacional (Ley de Conway aplicada deliberadamente). Un monolito habría simplificado la operación pero forzado acoplamiento de equipos y de releases.

8.2 Arquitectura orientada a eventos (EDA)

Qué se implementó. Un topic compartido eciexpress_events en Azure Service Bus. Cada servicio publica con subject = routing key (servicio.entidad.accion) y consume desde su suscripción propia (orders-service, products-service, financial-service, fulfillment-service, notifications-service — ver servicebus-config.json). El filtrado por dominio se hace con reglas de la suscripción (en producción, reglas SQL aprovisionadas "vía Terraform" según comentarios de service-bus.service.ts) o con routing keys exactas

en el consumidor (orders). La coreografía de la saga de venta y de la devolución corre íntegramente sobre estos eventos.

Ventajas:

- **Desacoplamiento temporal:** fulfillment y notifications procesan a su ritmo; si están caídos, la suscripción retiene los mensajes (TTL 1 h en el emulador local; MaxDeliveryCount: 10 y dead-lettering en las suscripciones).
- **Extensibilidad:** agregar un consumidor nuevo (p. ej. reporting histórico) no toca a los productores.
- **Auditabilidad natural:** cada hecho de negocio queda materializado como evento con sobre estándar y correlación.
- **Coreografía sin orquestador:** no existe un “saga orchestrator” central que sea punto único de falla; cada servicio reacciona a los hechos que le competen.

Desventajas:

- **Difícil razonamiento global:** el flujo completo de una venta atraviesa ~10 eventos; mitigado con catálogos por servicio y (ahora) con este documento.
- **Entrega al-menos-una-vez:** duplicados y reordenamientos posibles; mitigado con idempotencia sistemática.
- **Contratos frágiles si no se gobiernan:** ya ocurrió una deriva (financial publicaba walletId sin userId y notifications omitía la notificación — documentado como advertencia en INTEGRACION-EVENTOS.md); mitigado con checklists de integración.
- **Depuración distribuida:** mitigada con correlationId en el sobre y el panel de flujo de eventos de reporting (GET /kpis/events).

Por qué se eligió. Los hechos del dominio (pago aprobado, entrega confirmada, QR expirado) son inherentemente asíncronos y con múltiples interesados. Orquestarlos con llamadas REST encadenadas habría acoplado disponibilidad (“si notifications está caído, no puedes vender”) y roto la autonomía de equipos.

8.3 API Gateway

Qué se implementó. Un gateway hecho a medida en NestJS (no un producto de mercado) con responsabilidades acotadas: throttling (60 req/60 s por uid o IP), verificación del ID token de Firebase, enriquecimiento firebaseUid → identidad local con caché Redis (TTL 60 s), política de rutas (públicas / bloqueadas / semi-públicas), inyección de headers de identidad, reescritura de paths por servicio, proxy HTTP con

re-streaming del body crudo (crítico para la firma del webhook Wompi) y proxy de upgrade WebSocket con validación de token.

Ventajas: un solo punto de autenticación y CORS; los servicios no dependen de Firebase (solo identity y gateway); superficie pública mínima (Swagger y endpoints internos invisibles desde internet); throttling coherente multi-réplica; lógica de enriquecimiento reutilizada por HTTP y WS.

Desventajas: punto único de falla y de congestión (mitigable con réplicas — el estado compartido ya está en Redis); código propio que hay que mantener (frente a un Azure API Management, se aceptó el costo a cambio de control fino: raw body para Wompi, reescrituras específicas de fulfillment, proxy WS custom); el TODO documentado de autenticación Socket.IO (?token= en el upgrade vs auth.token del CONNECT) es complejidad heredada de esta decisión.

Por qué se eligió. Los requisitos concretos (webhook con body intacto, reglas de path muy específicas por servicio, enriquecimiento con caché, WS proxy con sustitución de userId verificado) se resolvían con menos fricción en código propio NestJS —el mismo stack del equipo— que configurando un producto externo.

8.4 Base de datos por servicio

Qué se implementó. Seis bases PostgreSQL independientes en Neon (identity, products, orders, financial, fulfillment, notifications). reporting no tiene BD propia (lee App Insights y, en solo lectura allowlisted, las BDs de orders/financial). Cada servicio gestiona su esquema con sus propias migraciones (Prisma Migrate en identity; migraciones TypeORM en products/orders/financial/fulfillment/notifications).

Ventajas: aislamiento de esquema y de fallos (una migración fallida no bloquea a otros); libertad de modelado por dominio (identity normaliza RBAC; orders guarda snapshot de ítems con precios congelados; financial usa bigint para dinero); el acoplamiento por datos queda imposible por construcción.

Desventajas: sin JOINS cross-dominio (resuelto con proyecciones y snapshot de datos en eventos); sin transacciones distribuidas (resuelto con saga + outbox); duplicación controlada (tiendas proyectadas en 4 servicios); la analítica de negocio cross-servicio es limitada — la excepción de lectura de reporting es el compromiso pragmático adoptado.

Por qué se eligió. Es la única opción coherente con microservicios autónomos: una BD compartida habría reintroducido el acoplamiento de esquemas entre equipos que la descomposición buscaba eliminar.

8.5 Estilos internos

- **orders-service — Hexagonal (Ports & Adapters):** application/ (casos de uso y puertos), domain/ (modelos y máquina de estados puros), infrastructure/ (clientes HTTP, mensajería, persistencia TypeORM), presentation/ (controllers y gateway WS). Los puertos (order.repository.ts, event-publisher.ts, financial.port.ts, products.port.ts, identity.port.ts) permiten sustituir infraestructura sin tocar casos de uso — demostrado por la migración de broker y por el products.mock.client.ts.
- **fulfillment-service — por dominios con capas http/domain/infra:** codes/, deliveries/, qr/, expiration/, events/handlers, outbox/, proyecciones separadas.
- **products-service — feature modules NestJS:** features/{products, categories, promotions, inventory, cart, stores, health, audit} con controllers/services/repositories/entities por feature, y mensajería transversal en src/messaging/.
- **identity-service — módulos por agregado** (users, roles, stores, auth, internal, audit, pqrs, outbox) con guards/decoradores/pipes transversales en common/.
- **Frontend — SPA por páginas y servicios:** pages/{landing, login, home, store, cart, orders, fulfillment, messages, user, vendor, admin, info}, capa services/*.ts de acceso HTTP tipado, context/ y hooks/ para estado, lib/ para contratos compartidos (p. ej. orders-api.ts).

9. Diagrama C4 de contexto

Explicación del diagrama

El diagrama de contexto muestra a ECIEXPRESS como un único sistema desde la perspectiva de sus usuarios y dependencias externas. En la parte superior aparecen los cuatro roles del sistema — los mismos cuatro valores del enum SystemRole de identity-service (BUYER, VENDOR, ADMIN, ANALYST) — cada uno interactuando con la plataforma por HTTPS (y WebSocket seguro para chat/notificaciones en vivo).

Alrededor del sistema se observan las siete dependencias externas reales del código:

- **Firebase Authentication** tiene una doble relación: el usuario final se autentica directamente contra Firebase desde el navegador (SDK web), y la plataforma verifica esos tokens del lado servidor con el Admin SDK. Esto refleja la regla de oro del proyecto: las contraseñas nunca tocan la infraestructura propia.
- **Wompi** es el único externo que llama hacia adentro (webhook de resultado de pago), razón por la cual esa ruta es la única POST pública del perímetro y está autenticada por firma criptográfica en lugar de JWT.
- **Gmail API, Meta WhatsApp Cloud API y Twilio** son los proveedores de los canales de notificación de salida (correo, WhatsApp, SMS).
- **Hunyan3D API** es el servicio de IA (desplegado sobre GPU NVIDIA A10G en AWS EC2, fuera de Azure) que convierte las fotos de un producto en un modelo 3D GLB para la experiencia de catálogo.
- **Azure Application Insights** recibe la telemetría de toda la plataforma y habilita el centro de monitoreo interno.

El lector debe retener de esta vista que ECIEXPRESS es un sistema autocontenido en su lógica de negocio (ningún externo decide estados de pedido, stock ni saldos) y que sus integraciones externas son de tres tipos: identidad (Firebase), dinero (Wompi) y comunicación/IA/telemetría (resto).

10. Diagrama C4 de contenedores

Explicación del diagrama

Esta vista descompone ECIEXPRESS en sus contenedores desplegables y muestra las tres capas de comunicación del sistema.

Capa de entrada. La SPA (frontend/ecixpress, servida por Azure Static Web Apps) habla exclusivamente con el API Gateway, usando prefijos por servicio (/identity/..., /products/..., etc.) que el gateway reescribe al path interno de cada microservicio. Hay dos excepciones deliberadas y seguras: el navegador descarga directamente de Blob Storage las imágenes y modelos 3D públicos (evita pasar binarios por el gateway), y habla directamente con Firebase para el login (el token resultante es lo único que viaja al backend). El gateway es el único contenedor con ingress externo: en el compose local “Solo el gateway se expone al host”, y en producción el resto de Container Apps usan FQDN internos (*.internal.azurecontainerapps.io, según gateway/.env.example).

Capa de servicios. Los siete microservicios NestJS aparecen con su puerto, su responsabilidad y su tecnología de persistencia. Se distinguen dos servicios con WebSocket (orders para chat, notifications para la campana de notificaciones), que el gateway proxya validando el token en el upgrade. reporting-service es el único sin base de datos propia: su “estado” son las consultas KQL a Application Insights y las lecturas read-only permitidas a las BDs de orders/financial.

Capa de datos y mensajería. Cada servicio tiene su base PostgreSQL en Neon (database-per-service estricto). El topic eciexpress_events de Azure Service Bus centraliza todos los eventos de dominio: los productores publican con el routing key como subject y cada consumidor lee de su propia suscripción con filtros por dominio — así un mensaje se entrega una vez a cada servicio interesado, con reintentos (MaxDeliveryCount: 10) y dead-letter queue por suscripción. Redis cumple dos funciones para el gateway (caché de identidad y rate limiting distribuido) y Blob Storage guarda todos los binarios del sistema.

Las flechas REST internas (orders→financial para la comisión de hora pico, orders→identity para validar tokens del chat, products→identity para reconciliar tiendas, gateway→identity para el enriquecimiento) muestran el patrón del principio 3: los hechos van por el bus, las preguntas van por REST.

11. Diagramas de componentes

Se presenta un diagrama de componentes (C4 nivel 3) por cada contenedor de backend.

11.1 API Gateway

El diagrama muestra el gateway como una tubería de decisión por la que pasa cada request. El GatewayThrottlerGuard actúa antes que todo (usa el uid del JWT sin verificar únicamente como clave de conteo — la verificación real ocurre después). El ProxyController consulta a RouteConfig, que concentra la política de superficie pública: qué se bloquea con 404 (endpoints internos service-to-service, Swagger de los downstream, /notifications/send y /notifications/wakeup), qué es público sin token (health checks, catálogo de tiendas de solo lectura con lista precisa para no abrir /stores/my, el webhook de Wompi, las imágenes de QR/entrega que consumen los correos y WhatsApp, y el modelo 3D público), y el caso semi-público de POST /identity/auth/sync-profile (token verificado pero sin exigir perfil local, porque es justamente el endpoint que lo crea).

Para rutas protegidas, `FirestoreService` verifica el token y `EnrichmentService` traduce `firebaseUid` a `{ userId, roles, storeId, status }` con caché de 60 segundos — el TTL corto es la estrategia de invalidación: un cambio de rol o una suspensión demora máximo un minuto en aplicarse en el perímetro. `HeaderInjector` implementa la defensa anti-spoofing (borra los headers de identidad que traiga el cliente e inyecta los reales, distintos por servicio destino) y `ProxyService` reenvía preservando el body crudo. `WsProxyService` replica esta misma cadena para los upgrades de `WebSocket`. El `AcmeController` delata el soporte de emisión de certificados para dominio propio vía `HTTP-01`.

11.2 identity-service

`Identity-service` aparece organizado en cuatro franjas. En presentación, siete controllers separan el API pública del API interna: `InternalController` existe solo para consumo entre servicios (el gateway lo bloquea desde internet) y es la pieza que soporta tanto el enriquecimiento del gateway (`users/by-firebase/:uid`) como las validaciones síncronas de otros servicios (disponibilidad de tienda, staff, perfil).

En aplicación, cada agregado tiene su servicio; destaca `ClosureSchedulerService`, el job que materializa los cierres temporales programados: cuando un cierre vence, actualiza el estado de la tienda y publica los eventos correspondientes con `performedBy = null` (cambio automático, no humano).

En transversales está la cadena de seguridad propia del servicio (que, a diferencia del resto de microservicios, sí verifica el token de Firebase por sí mismo — `identity` es la autoridad): `FirestoreAuthGuard` → carga de perfil → `PermissionsGuard` contra los permisos del decorador; `SessionService` implementa sesión única con TTL de 8 horas validando el header `X-Session-Id` que el gateway reenvía.

En infraestructura, el trío `OutboxService` → `OutboxWorker` → `ServiceBusPublisherService` implementa la publicación confiable: el evento se guarda en `outbox_events` en la misma transacción que el cambio de negocio; el worker lo publica con reintentos exponenciales y el publisher usa `Managed Identity` en producción. El routing key se deriva automáticamente de `aggregateType + eventType` (p. ej. `("Store", "StoreStatusChanged")` → `identity.store.status_changed`), lo que evita inconsistencias manuales.

11.3 products-service

products-service es el contenedor con más features. La mitad superior muestra el API REST del catálogo que consumen el frontend y los vendedores: productos (con soporte multipart para crear el producto junto con sus fotos), categorías jerárquicas, promociones con cálculo de precio efectivo, y consulta de movimientos de inventario. El endpoint público GET `/:id/model3d` sirve el modelo GLB al visor 3D del frontend.

La feature Cart no expone REST: vive enteramente de eventos. Mantiene una proyección del carrito (con la invariante `cartId == orderId`), cotiza de forma autoritativa (`products.cart.priced` con precios de lista, promoción aplicada y totales) y gestiona el ciclo de reserva de stock: al llegar `order.order.created` reserva unidad por unidad (`reserved_stock`); si todas las líneas quedan reservadas publica `reservation_confirmed`, si alguna falla publica `reservation_rejected` con el disponible real; al confirmar el pedido consume la reserva (descuenta stock físico) y al cancelar decide — usando el flag `wasSold` del evento — si libera la reserva o restituye stock ya descontado. Cuando el disponible cae al mínimo, emite `product.inventory.low_stock`.

La feature Stores ilustra el patrón de proyección con reconciliación: cualquier `identity.store.*` dispara un pull del snapshot completo desde el endpoint interno de `identity`, porque los payloads de esos eventos no garantizan el estado completo.

`ProductMediaService` y `ModelRetryScheduler` forman el pipeline 3D: validación de vistas (solo-frontal o tres vistas), subida a Blob, llamada a la API Hunyuan3D con timeout de 10 minutos, persistencia del GLB y reintentos programados con estado de progreso visible (`modelGenerationStatus/Progress/Error` en la entidad).

11.4 orders-service

El hexágono central contiene la lógica que hace de orders el cerebro transaccional del sistema. `order.models.ts` define la máquina de estados pura (13 estados, tabla `ORDER_TRANSITIONS` explícita, transiciones validadas con historial de actor y razón), y `OrdersService` implementa los casos de uso sobre los seis puertos — nunca sobre implementaciones concretas.

A la izquierda, los adaptadores de entrada: el API REST del ciclo de vida (nótese que el carrito es un pedido en DRAFT: POST `/orders/draft` lo crea, POST `/:id/items` lo modifica y POST `/:id/checkout` lo lanza a pago) y el gateway Socket.IO del chat, que autentica el handshake contra `identity` (`/auth/validate`), organiza salas por usuario y por conversación, y reacciona al evento interno `session:revoked` desconectando todos los sockets de un usuario desactivado — el eslabón en tiempo real de `identity.user.deactivated`.

A la derecha, los adaptadores de salida: persistencia TypeORM (con `outbox_events` y `processed_events` como tablas de soporte de mensajería), el ciclo `outbox→worker→ServiceBus` para publicar, el consumidor idempotente de los 11 `routing keys` de entrada, y los clientes HTTP hacia `financial` (cotización de hora pico con degradación a recargo 0), `products` e `identity`. `RealtimeHubService` desacopla los manejadores de eventos del gateway WS: los consumidores publican en el hub y el gateway emite a las salas.

11.5 financial-service

El diagrama separa las dos caras de `financial`. La cara billetera/recargas (arriba a la izquierda): el usuario consulta su saldo, inicia una recarga eligiendo método (tarjeta tokenizada por Wompi — nunca datos crudos de tarjeta —, Nequi, PSE o Bancolombia) y Wompi responde de forma asíncrona por el webhook, cuyo controller verifica la firma SHA-256 calculada sobre el body crudo (por eso el gateway reenvía el buffer intacto). Aprobada la recarga, se acredita la billetera, se genera recibo PDF y se publica `financial.wallet.topup.approved`.

La cara escrow de ventas (centro): `TransactionsService.handleOrderCreated` es el corazón — idempotente por restricción única sobre `order_id` (una carrera de eventos duplicados termina en violación 23505 tratada como éxito), valida tienda y billetera, calcula el desglose (comisión de plataforma sobre el valor de productos; el recargo de hora pico viene ya fijado por `orders` para honrar “precio visto = precio cobrado”), debita y persiste la transacción en estado HELD — todo dentro de una transacción de base de datos. El dinero permanece retenido hasta `fulfillment.delivery.confirmed`, que lo libera (RELEASED) acreditando el neto al negocio, o hasta una cancelación/devolución, que aplica la política de reembolso indicada en el evento (100%, HALF_PRODUCTS_ONLY o NO_REFUND).

La proyección local de tiendas guarda la configuración financiera por tienda (porcentaje de comisión, franja y recargo de hora pico, cuenta de payout) que administra el rol ADMIN.

11.6 fulfillment-service

`fulfillment` materializa la credencial de retiro. El diagrama muestra su regla más importante en la franja de eventos: `order.order.confirmed` solo proyecta el pedido; el código se genera al llegar `order.order.ready_for_pickup` — es decir, el QR existe cuando el pedido está físicamente listo. La generación produce dos representaciones del mismo secreto: el token opaco (que codifica el QR y la URL PNG pública-inadivisible que insertan los correos/WhatsApp) y el código corto tecleable en base32 sin caracteres ambiguos (A7K9-P2MX), para cuando la cámara falla.

OrderHandler exhibe el mecanismo de concurrencia más fino del sistema: un advisory lock transaccional de PostgreSQL sobre la idempotencyKey serializa entregas casi simultáneas del mismo evento — el comentario del código documenta el bug que esto cierra (dos entregas concurrentes generaban dos QR duplicados al pasar ambas el chequeo de idempotencia antes de que cualquiera confirmara).

En el flujo de entrega, el vendedor (autorizado contra la proyección local de staff, alimentada por eventos de identity) valida el código y confirma; fulfillment.delivery.confirmed es el evento que aguas abajo mueve el pedido a DELIVERED y libera el dinero retenido — por eso nace del outbox transaccional (“garantiza que delivery.confirmed (que mueve dinero) nunca se pierda”). La entrega manual y el registro de fallos cubren los caminos alternos, y ExpirationService cierra el ciclo expirando códigos vencidos (lo que desencadena la cancelación con política NO_REFUND).

11.7 notifications-service

notifications es un consumidor terminal: no publica eventos de negocio. Su núcleo traduce hechos del bus en comunicaciones humanas. NotificationCatalog es el componente clave: mapea cada routing key a título/cuerpo, destinatario y canales — así, publicar el evento de dominio correcto basta para que “ocurran” las notificaciones (Opción A de la guía de integración); para necesidades a la medida existe el comando notification.send.requested (o el REST interno POST /notifications/send, bloqueado en el perímetro).

RecipientsService explica cómo el servicio conoce a la gente: construye contactos locales a partir de identity.user.registered (email) y identity.user.profile_updated (teléfono — requisito para WhatsApp/SMS), y aprende el mapeo storeId → ownerId de identity.store.created para poder “notificar a la tienda”.

Los cuatro canales implementan una interfaz común (patrón Strategy): correo por Gmail API con OAuth2 (el código documenta el lock explícito que garantiza un solo refresh de token en vuelo — corrige errores invalid_grant bajo concurrencia), WhatsApp por Meta Cloud API (con soporte de imagen: el QR de retiro viaja como media), SMS por Twilio, y tiempo real por Socket.IO con sala por usuario. Sin credenciales configuradas, un canal opera en sandbox (registra sin entregar); si el usuario desactivó el canal en sus preferencias, la entrega queda SKIPPED sin fallar el evento. La bandeja in-app siempre se persiste, esté o no conectado el usuario — el historial es consultable por REST.

11.8 reporting-service

reporting-service es el centro de monitoreo de la plataforma para los roles ADMIN/ANALYST (el RolesGuard valida el header de rol inyectado por el gateway). Su particularidad arquitectónica es que no posee estado propio: todos sus datos provienen de cuatro familias de fuentes — (1) telemetría en Application Insights consultada con KQL parametrizado y con ventanas acotadas ("mínimo 1h, máximo 30 días. Acota el costo/latencia de la consulta KQL"), (2) métricas de Azure Monitor (incluido el namespace de Service Bus), (3) lecturas de solo lectura a las bases de orders y financial mediante un catálogo cerrado de consultas parametrizadas con timeout, y (4) planos de control: GitHub Actions y Azure Resource Manager para responder "¿qué versión está desplegada y cómo salió el último deploy?".

Con esas fuentes, los tres controllers arman el panel del frontend: overview (salud técnica global), latency (p95 por servicio), events (el flujo de eventos entre microservicios — la herramienta operativa para detectar sagas atascadas), quick (indicadores de negocio), logs y deploy. El comentario del compose que describe a reporting como "consume el bus" está desactualizado: su package.json no incluye @azure/service-bus; observa el bus desde las métricas, no desde una suscripción (discrepancia registrada en Deuda Técnica).

12. Descripción de cada microservicio

Convenciones de las tablas: las rutas se muestran como las expone el servicio (el gateway antepone el prefijo /identity, /products, etc. y, para fulfillment, añade /api/v1). "REST interno" = llamadas servicio-a-servicio directas (no pasan por el gateway).

12.1 identity-service

Aspecto	Detalle
Propósito	Fuente de verdad de identidad, control de acceso (RBAC) y administración de puntos de venta
Responsabilidades	Perfiles vinculados a Firebase (firebaseUid), sesión única (8 h, X-Session-Id), roles y permisos granulares, CRUD de tiendas + horarios + cierres temporales + staff, imágenes de tienda y avatar (Blob), auditoría append-only, tickets PQRS, publicación confiable de eventos identity.*
Endpoints	GET /auth/validate; POST /auth/change-password; POST /auth/sync-profile; GET/PUT /users/me; PATCH /users/me/phone; POST /users/me/avatar; DELETE /users/me; GET /users, GET /users/:id, PATCH /users/:id/status, PATCH /users/bulk/status; roles: GET/POST /roles, GET /permissions, GET/PUT /roles:roleId/permissions, POST /users/:id/roles, POST

	/users/bulk/roles, DELETE /users/:id/roles/:roleId; tiendas: GET/POST /stores, GET /stores/available
Dependencias	Firebase Admin SDK (verificación de tokens), Azure Blob Storage, Azure Service Bus (publicación)
Tecnologías	NestJS 11, Prisma 7 (@prisma/adapters-pg), Zod, pino/pino-http, firebase-admin, @azure/service-bus, @azure/storage-blob, Application Insights. Puerto 3001
Base de datos	PostgreSQL propia (Neon): users, roles, permissions, user_roles, role_permissions, stores, store_staff, store_schedules, store_closures, user_sessions, outbox_events, audit_logs, pqrs, pqrs_messages
Eventos publicados	identity.user.registered, identity.user.profile_updated, identity.user.deactivated, identity.user.role_changed, identity.role.updated, identity.store.created, identity.store.updated, identity.store.status_changed, identity.store.schedule_changed, identity.store.temporarily_closed, identity.store.closure_cancelled, identity.store.closure_expired, identity.store.staff_changed
Eventos consumidos	Ninguno (decisión de MVP documentada: las validaciones cruzadas se hacen por REST interno)
Comunicación REST	Sirve: gateway (validate/enriquecimiento), orders (validación de token del chat), products (reconciliación de tiendas), financial/fulfillment vía proyecciones. No consume REST de otros
Comunicación asíncrona	Solo productor. Outbox transaccional → worker (backoff exponencial 2^retry, máx. 5) → topic eciexpress_events

12.2 products-service

Aspecto	Detalle
Propósito	Dueño del catálogo, precios/promociones e inventario; autoridad de cotización del carrito y de la reserva de stock
Responsabilidades	CRUD de productos con imágenes multivista y modelo 3D, categorías jerárquicas, promociones y precio efectivo, movimientos de inventario, proyección de carrito y cotización autoritativa, reserva/consumo/liberación/restitución de stock por pedido, alerta de stock bajo, réplica local de tiendas con reconciliación
Endpoints	Productos (@Controller(")): GET /, GET /store/:storeId, GET /store/:storeId/low-stock, GET /store/:storeId/category/:categoryId, GET /:id, GET /:id/model3d (público), POST /, POST /with-assets (multipart), PATCH /:id, PATCH /:id/activate
Dependencias	identity-service (REST interno de reconciliación), Hunyuan3D API (PRODUCT_AI_GENERATE_URL, timeout 10 min), Azure Blob Storage (contenedor products), Azure Service Bus

Tecnologías	NestJS 11, TypeORM, class-validator/transformer, multer, winston + nest-winston, @nestjs/terminus, @nestjs/schedule, axios. Puerto 3002
Base de datos	PostgreSQL propia: products (con stock, reserved_stock, min_stock, campos 3D), categorías, promociones, movimientos de inventario, proyección de carrito, réplica de tiendas, auditoría
Eventos publicados	products.cart.priced, products.return.priced, product.inventory.low_stock, product.inventory.reservation_confirmed, product.inventory.reservation_rejected, product.item.created
Eventos consumidos	order.cart.created, order.cart.item_changed, order.return.requested, order.order.created, order.order.confirmed, order.order.cancelled, identity.store.* (dispara reconciliación pull)
Comunicación REST	Sirve al frontend (catálogo) y a orders (ProductsHttpClient). Consume identity interno
Comunicación asíncrona	Productor y consumidor; suscripción products-service; en error transitorio lanza para que Service Bus reentregue (→ DLQ tras maxDeliveryCount)

12.3 orders-service

Aspecto	Detalle
Propósito	Dueño único del ciclo de vida del pedido y del canal de comunicación comprador-vendedor
Responsabilidades	Carrito (orden DRAFT) y cotización vía products, checkout con recargo de hora pico fijado con financial, máquina de estados de 13 estados con historial auditado, coordinación de la saga (pago + reserva de stock), cancelaciones con políticas de reembolso, devoluciones con evidencia y aprobación admin, calificaciones, chat en tiempo real y su historial
Endpoints	POST /orders (directo), POST /orders/draft, POST /orders/:id/items, POST /orders/:id/quote, POST /orders/:id/checkout, POST /orders/:id/returns, POST /orders/:id/returns/evidence, GET /orders/:id/returns/evidence/:refundId, POST /orders/:id/returns/approve
Dependencias	financial (REST GET /stores/:id/commission), products (REST catálogo + eventos), identity (REST GET /auth/validate para WS), Azure Blob (evidencia), Azure Service Bus
Tecnologías	NestJS 11, TypeORM, Socket.IO (@nestjs/websockets + platform-socket.io), axios, class-validator. Puerto 3000. Arquitectura hexagonal
Base de datos	PostgreSQL propia: orders, order_items, order_status_history, order_ratings, conversations, participants, messages, outbox_events, processed_events
Eventos publicados	order.order.created, order.order.confirmed, order.order.cancelled, order.order.ready_for_pickup, order.order.status_changed, order.chat.message.sent, order.cart.created, order.cart.item_changed, order.return.requested, order.return.confirmed

Eventos consumidos	financial.payment.processed
Comunicación REST	Sirve al frontend (pedidos y chat). Consume financial, products, identity
Comunicación asíncrona	Productor y consumidor; outbox (poll 5 s, 5 reintentos → FAILED); dedupe por processed_events

12.4 financial-service

Aspecto	Detalle
Propósito	Autoridad del dinero: billeteras, recargas, cobro con retención (escrow), comisiones, reembolsos, recibos y payouts
Responsabilidades	Billetera por usuario (creada al registrarse), recargas Wompi (CARD tokenizada, NEQUI, PSE, BANCOLOMBIA) con webhook firmado, débito atómico + transacción HELD al crearse el pedido, liberación al confirmar entrega (RELEASED + payout neto), reembolsos según política (100%, HALF_PRODUCTS_ONLY, NO_REFUND) y por devoluciones cotizadas, configuración financiera por tienda (comisión, hora pico, cuenta de payout), recibos PDF en Blob
Endpoints	GET /wallet; GET/POST /wallet/topups, GET /wallet/topups/pse-institutions, GET /wallet/topups/:id, GET /wallet/topups/:id/receipt-url; POST /webhooks/wompi (público vía gateway); GET /wallet/transactions, GET /wallet/transactions/:orderId/receipt-url; PATCH /stores/payout-account, GET /stores/:storeId/commission; GET /stores/payouts, GET /stores/payouts/earnings/:storeId; admin: PATCH /admin/stores/:storeId/commission-config, GET /admin/stores/:storeId/transactions, GET /admin/transactions
Dependencias	Wompi API (axios), Azure Blob (recibos), Azure Service Bus, gateway (webhook con raw body)
Tecnologías	NestJS 11, TypeORM, pdfkit, axios, class-validator. Puerto 3004
Base de datos	PostgreSQL propia: wallets (saldo bigint en centavos), wallet_topups (estado, wompi_transaction_id, wompi_response jsonb, recibo), order_transactions (desglose completo + held_at/released_at/refunded_at, único por order_id), stores (proyección + config financiera)
Eventos publicados	financial.payment.processed, financial.payment.failed, financial.payment.released, financial.refund.issued, financial.wallet.topup.approved, financial.wallet.topup.failed
Eventos consumidos	identity.user.registered
Comunicación REST	Sirve a orders (cotización de comisión) y al frontend (billetera). Consume Wompi

Comunicación asíncrona	Productor y consumidor; idempotencia por unicidad de order_id (PG 23505 tratado como duplicado benigno)
-----------------------------------	---

12.5 fulfillment-service

Aspecto	Detalle
Propósito	Logística de entrega: credencial de retiro (QR + código corto), validación, confirmación/fallo de entrega y expiración
Responsabilidades	Generar el código al quedar el pedido listo (ready_for_pickup), servir el PNG del QR (público-inadivivable) e imágenes para plantillas de notificación, validar códigos, confirmar entregas (QR o manual), registrar fallos, expirar códigos (job) y avisar próximos a vencer, invalidar por cancelación, mantener proyecciones de pedido y de staff
Endpoints	(internamente bajo setGlobalPrefix('api/v1') con exclusiones) GET /fulfillment/orders/:orderId/code, POST /fulfillment/codes/validate, POST /fulfillment/codes/confirm, POST /fulfillment/orders/:orderId/manual-delivery, POST /fulfillment/orders/:orderId/delivery-failure, GET /fulfillment/orders/:orderId, GET /fulfillment/stores/:storeId/deliveries; públicos fuera de /api/v1: GET /fulfillment/qr:file, GET /fulfillment/delivery/:orderId, GET /fulfillment/notification-image/{confirmed,cancelled,qr-expired}/:orderId, GET /fulfillment/notification-image/{payment-processed,refund-issued}/:orderId/:amount
Dependencias	Azure Service Bus, PostgreSQL, librerías qrcode + sharp
Tecnologías	NestJS 11, TypeORM, @nestjs/schedule + cron, nestjs-pino, uuid. Puerto 3005
Base de datos	PostgreSQL propia (fulfillment_db): pickup_codes, deliveries, order_projection, store_staff_projection, outbox_events, processed_events, audit_logs
Eventos publicados	fulfillment.qr.generated, fulfillment.qr.expired, fulfillment.qr.expiring_soon, fulfillment.delivery.confirmed, fulfillment.delivery.failed
Eventos consumidos	order.order.confirmed (proyección), order.order.ready_for_pickup (genera código), order.order.cancelled (invalida), identity.store.created, identity.store.staff_changed
Comunicación REST	Sirve al frontend (comprador ve su código; vendedor valida/confirma) y a notificaciones (URLs de imágenes). No consume REST
Comunicación asíncrona	Productor y consumidor; advisory locks (pg_advisory_xact_lock) + processed_events para idempotencia bajo concurrencia

12.6 notifications-service

Aspecto	Detalle
Propósito	Convertir eventos de dominio en notificaciones multicanal y mantener la bandeja in-app
Responsabilidades	Catálogo evento→notificación (destinatario, texto, canales), contactos locales (email/teléfono) y mapeo tienda→dueño, envío por EMAIL/WHATSAPP/SMS/REALTIME con sandbox y SKIPPED por preferencia, plantillas HTML por routing key, bandeja persistente con no-leídas, preferencias por usuario, reintentos de entrega, keep-alive de servicios
Endpoints	GET /notifications (paginada, x-user-id), GET /notifications/unread-count, PATCH /notifications/:id/read, POST /notifications/read-all, POST /notifications/send (interno, bloqueado en gateway), GET/PATCH /preferences, POST /wakeup (interno, bloqueado)
Dependencias	Gmail API (OAuth2), Meta WhatsApp Cloud API v25.0, Twilio, Azure Service Bus, fulfillment (URLs de imágenes QR/entrega en mensajes)
Tecnologías	NestJS 11, TypeORM, Socket.IO, googleapis, nodemailer (soporte), axios, @nestjs/schedule + cron. Puerto 3006
Base de datos	PostgreSQL propia: notificaciones, entregas por canal (estado), preferencias, contactos/receptores, deduplicación
Eventos publicados	Ninguno de negocio
Eventos consumidos	27 routing keys: 9 de identity, 5 de order (incl. chat), 5 de fulfillment, 6 de financial, product.inventory.low_stock y notification.send.requested (bindings identity.#, order.#, fulfillment.#, financial.#, product.#, notification.#)
Comunicación REST	Sirve al frontend (bandeja/preferencias). Consume APIs de terceros
Comunicación asíncrona	Consumidor puro; idempotencia por idempotencyKey o routingKey + id de entidad

12.7 reporting-service

Aspecto	Detalle
Propósito	Centro de monitoreo y analítica para ADMIN/ANALYST: KPIs técnicos, indicadores de negocio, salud y estado de despliegues
Responsabilidades	Overview técnico (volumen, tasa de error, p95, top excepciones), latencia por servicio/minuto, flujo de eventos entre servicios, consultas rápidas de negocio (allowlisted, read-only), logs por servicio, salud de los 7 servicios, estado de deploys (GitHub + revisiones ACA), métricas del Service Bus
Endpoints	GET /kpis/overview, GET /kpis/latency, GET /kpis/events, GET /kpis/health, GET /kpis/quick, GET /kpis/quick/:key, GET /kpis/logs/:service, GET /kpis/deploy/:service, GET /kpis/servicebus

Dependencias	Application Insights (KQL), Azure Monitor Metrics, orders_db/finacial_db read-only, GitHub API, Azure Resource Manager
Tecnologías	NestJS 11, @azure/monitor-query-logs, @azure/monitor-query-metrics, @azure/arm-appcontainers, pg directo. Puerto 3000
Base de datos	Sin BD propia (pools read-only con statement_timeout 8 s, max 3 conexiones)
Eventos publicados / consumidos	Ninguno (no incluye @azure/service-bus; observa el bus vía métricas)
Comunicación REST	Sirve al frontend (rol ADMIN/ANALYST). Consume /health de los 7 servicios y APIs de Azure/GitHub
Comunicación asíncrona	No participa del bus

12.8 API Gateway (contenedor de soporte)

Aspecto	Detalle
Propósito	Única puerta de entrada: autenticación, autorización perimetral, throttling y enrutamiento HTTP/WS
Responsabilidades	Verificar ID tokens Firebase; enriquecer identidad (caché 60 s); rechazar usuarios no ACTIVE y perfiles inexistentes (403 PROFILE_NOT_FOUND); strip+inyección de headers; correlación; rate limiting 60/60 s; políticas de rutas públicas/bloqueadas; reescritura de paths; proxy raw-body (webhook Wompi); proxy WS con token; challenge ACME
Endpoints propios	GET /health, GET /.well-known/acme-challenge/:token, @All('*path') (proxy)
Dependencias	Firebase Admin, Redis (opcional con degradación), identity-service, los 7 servicios downstream
Tecnologías	NestJS 11, http-proxy, ioredis, @nestjs/throttler (storage custom), firebase-admin. Puerto 3000 (único publicado)
Base de datos	Ninguna (estado efímero en Redis)
Eventos	No participa del bus

13. Comunicación síncrona mediante REST

13.1 Modelo general

Toda la comunicación cliente→backend es REST/JSON sobre HTTPS a través del gateway, con dos excepciones de solo-lectura directa (Blob Storage para binarios públicos y Firebase para login). Entre servicios existe además un conjunto acotado de llamadas REST **internas** (directas al FQDN interno del servicio, sin pasar por el gateway), reservadas para preguntas que necesitan respuesta inmediata y autoritativa.

Convenciones observadas en el código:

- **Prefijo por servicio** en el gateway → reescritura al path interno (/products/categories → /categories). fulfillment es el caso especial (/api/v1 interno, con qr/ y delivery/ excluidos).
- **Identidad por headers inyectados:** x-user-id, x-user-role, x-user-store (según servicio destino), más x-correlation-id siempre. Para rutas de identity se reenvía el Authorization: Bearer original y el X-Session-Id.
- **Idempotencia en mutaciones:** Idempotency-Key pasa intacto hacia orders (clave persistida en la orden para evitar pedidos duplicados).
- **Validación:** ValidationPipe global con whitelist + forbidNonWhitelisted (orders y otros), Zod en identity.
- **Documentación:** Swagger en cada servicio (/api), **no expuesto** por el gateway.
- **Errores:** filtros globales de excepción con formato JSON homogéneo; códigos de negocio en el gateway (PROFILE_NOT_FOUND, USER_INACTIVE).

13.2 Matriz de llamadas REST internas (servicio→servicio)

Origen	Destino Endpoint	Propósito	Resiliencia observada
gateway	identity GET /internal/users/by-firebase/:uid	Enriquecimiento firebaseUid → {userId, roles, storeId, status}	Caché Redis TTL 60 s; sin Redis llama siempre; 404 → 403 PROFILE_NOT_FOUND
gateway	identity GET /auth/validate (reenvío del Bearer del cliente)	Validación completa con permisos (rutas /identity/*)	—
orders	financial GET /stores/:storeId/commission?amount=	Fijar recargo de hora pico en el checkout ("precio visto = precio cobrado")	Timeout 6 s; ante fallo devuelve recargo 0 y no bloquea el checkout; el cobro real lo recalcula financial

orders	identity	GET /auth/validate	Autenticar el handshake del WebSocket de chat	Conexión rechazada si falla
orders	products	Cliente ProductsHttpClient (catálogo para armar líneas del carrito)	Datos de producto al agregar ítems	Existe ProductsMockClient para desarrollo
products	identity	Endpoint interno de tienda (vía StoreSyncService)	Reconciliación pull de la réplica de tiendas al llegar	Error transitorio → se relanza para reentrega del evento identity.store.*
notifications	los 7 servicios	GET /health (wakeup)	Despertar servicios con cold start y verificar UP trigger	Timeouts diferenciados (5 s / 90 s espera / 20 s verificación)
reporting	los 7 servicios	GET /health	Panel de salud	Timeout por servicio
financiamiento	identity	(cliente identity.client.ts en common/clients)	Apoyo de validación/consulta	<i>Información pendiente de confirmar con el equipo de desarrollo (uso exacto en runtime)</i>

13.3 API pública por el gateway (resumen de superficie)

Prefijo	Autenticación	Ejemplos representativos
/identity/*	Bearer Firebase reenviado (identity valida) + X-Session-Id	POST /identity/auth/sync-profile (semi-pública), GET /identity/users/me, GET /identity/stores (pública), POST /identity/pqrs
/products/*	Headers inyectados	GET /products/store/:id, GET /products/:id/model3d (pública), POST /products/with-assets
/orders/*	Headers inyectados (+ Idempotency-Key)	POST /orders/draft, POST /orders/:id/checkout, GET /orders/history
/financiamiento/*	Headers inyectados; webhook público	GET /financiamiento/wallet, POST /financiamiento/wallet/topups, POST /financiamiento/webhooks/wompi (pública, firma)
/fulfillment/*	Headers inyectados; QR/imágenes públicos	GET /fulfillment/orders/:id/code, POST /fulfillment/codes/confirm, GET /fulfillment/qr/:file (pública)

/notifications/*	Headers inyectados	GET /notifications, PATCH /notifications/preferences — send y wakeup bloqueados
/reporting/*	Headers inyectados + rol ADMIN/ANALYST	GET /reporting/kpis/overview

Rutas bloqueadas (404 incondicional): /identity/internal/*, /notifications/send, /notifications/wakeup, /<servicio>/api (Swagger).

14. Comunicación asíncrona mediante eventos

14.1 Infraestructura de mensajería

Elemento	Valor (evidencia)
Broker	Azure Service Bus (@azure/service-bus en los 6 servicios participantes)
Entidad	Topic eciexpress_events
Direccionamiento	subject del mensaje = routing key servicio.entidad.accion; copia en applicationProperties.routingKey
Suscripciones	Una por consumidor: orders-service, products-service, financial-service, fulfillment-service, notifications-service (servicebus-config.json); LockDuration PT1M, MaxDeliveryCount 10
Filtrado	Reglas de la suscripción (en producción, reglas SQL — comentario en servicebus.service.ts: “el filtro por dominio vive en la regla SQL de la subscription (Terraform)”); orders filtra además por lista exacta en código
Autenticación	Producción: Managed Identity (DefaultAzureCredential + SERVICE_BUS_FULLY_QUALIFIED_NAMESPACE); local: connection string al emulador oficial (mcr.microsoft.com/azure-messaging/servicebus-emulator respaldado por Azure SQL Edge)
Formato	JSON plano (sin subnodo payload), contentType: application/json
Historia	Migrado desde RabbitMQ/CloudAMQP (exchange topic homónimo); la API interna de los servicios se conservó (publish/consume) para no tocar lógica de negocio

14.2 El sobre estándar

Todo evento publicado lleva cinco metadatos añadidos automáticamente por el outbox del productor, al mismo nivel que los campos de negocio:

Campo	Tipo	Función
eventVersion	number (hoy 1)	Versionamiento del contrato
source	string	Servicio productor ("orders-service", "identity-admin-service", ...)
correlationId	uuid \	null
occurredAt	ISO-8601 UTC	Momento de emisión
idempotencyKey	uuid	Deduplicación en consumidores

Reglas transversales: montos **en centavos de COP como enteros** (650000 = \$6.500); identificadores de negocio al primer nivel (orderId, cartId, storeId, userId); reason textual en eventos de fallo.

14.3 Publicación confiable: Transactional Outbox

- La capa de aplicación entrega el evento al servicio de outbox, que lo guarda en la tabla outbox_events con estado PENDING y el sobre estándar, dentro de la misma transacción de base de datos que el cambio de negocio.
- Un worker hace polling de los eventos PENDING y los publica al topic eciexpress_events de Azure Service Bus (subject = routing key).
- Si la publicación tiene éxito, el evento se marca PUBLISHED; si falla, se reintenta con backoff y, agotados los reintentos, queda FAILED para revisión manual.

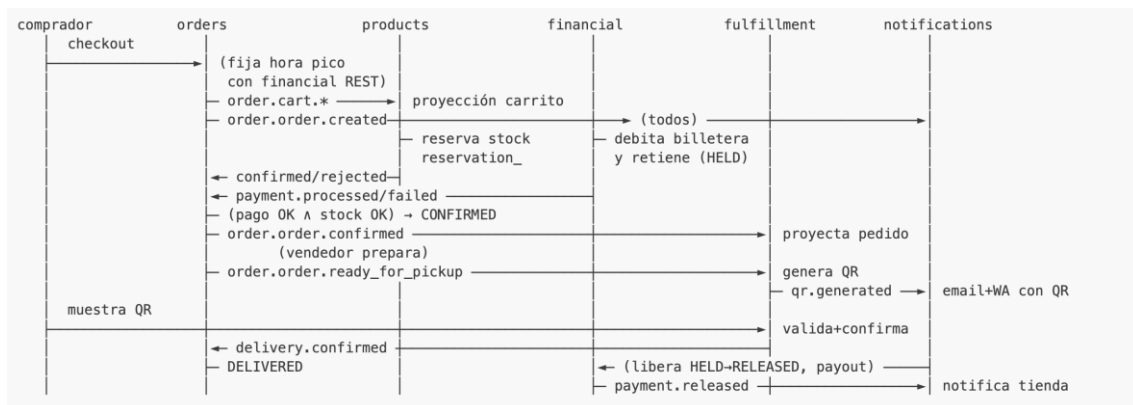
Implementaciones verificadas: **identity** (Prisma; backoff exponencial $2^{\text{retryCount}}$, máx. 5, nextRetryAt), **orders** (TypeORM; poll cada 5 s, 5 reintentos), **fulfillment** (TypeORM; "ningún evento se publica directo... garantiza que delivery.confirmed (que mueve dinero) nunca se pierda"). products y financial publican mediante sus publishers compartidos (SharedEventPublisherService / EventPublisherService); no se encontró tabla outbox en ellos — la publicación es directa al bus (ver *Deuda Técnica DT-06*).

14.4 Consumo confiable: idempotencia y reentrega

Mecanismo	Servicios	Detalle
Tabla processed_events por idempotencyKey	orders, fulfillment, notifications (dedupe equivalente)	Cada clave se procesa una sola vez

Restricción única de negocio	financiamiento (order_id único en order_transactions)	El duplicado pierde la carrera y se ignora (PG 23505)
Advisory lock transaccional	fulfillment (pg_advisory_xact_lock(hashtext(idempotencyKey)))	Serializa entregas concurrentes del mismo evento
Reentrega del broker	products (y patrón general)	Ante error transitorio se lanza para abandonar el mensaje → reintento; tras maxDeliveryCount → DLQ
Ack-siempre	orders (consumer)	Completa el mensaje aunque el handler falle, para evitar <i>poison loops</i> ; el error queda logueado

14.5 Flujo maestro de la saga de venta (coreografía)



Reglas de confluencia verificadas en orders.service.ts:

- **Pedido digital (wallet/tarjeta):** CONFIRMED requiere payment.processed y reservation_confirmed (flag stockReserved); el orden de llegada es indistinto (ambos manejadores reintentan la confirmación).
- **Pedido en efectivo ("Option C"):** checkout → CREATED; reservation_confirmed → CONFIRMED directamente (el cobro es contra entrega).
- **reservation_rejected** → el pedido falla y se compensa (liberación de retención vía order.order.cancelled/fallo).
- **Cancelaciones:** order.order.cancelled lleva wasSold (¿restituir stock físico o liberar reserva?) y refundPolicy (HALF_PRODUCTS_ONLY si cancela el cliente antes de listo; NO_REFUND si expiró el QR; ausente = 100%).

14.6 Flujo de devolución

1. Comprador solicita (POST /orders/:id/returns, con motivo y evidencia opcional en Blob) → order.return.requested.
2. products cotiza contra su proyección → products.return.priced (monto autoritativo).
3. Si el pedido ya fue entregado, orders lo pasa a RETURN_PENDING_APPROVAL (y reabre el chat para la gestión); un ADMIN aprueba o rechaza.
4. Aprobada: orders emite order.return.confirmed con el monto → financial acredita el reembolso → financial.refund.issued → notifications informa al comprador; el estado final es RETURNED o PARTIALLY_RETURNED.

15. Comunicación en tiempo real mediante WebSockets

15.1 Topología

Existen **dos servidores Socket.IO** independientes detrás del gateway, más un proxy de upgrade en el gateway que autentica antes de reenviar:

Canal	Servidor	Path	Namespace	Propósito
Chat de pedidos	orders-service	/orders/socket.io	/communication	Mensajería comprador-vendedor por conversación, typing, lectura, actualización de estado del pedido en vivo
Notificaciones	notifications-service	/notifications/socket.io	(default)	Campana in-app: push de notificaciones a todas las pestañas/dispositivos del usuario

15.2 Autenticación del canal

- 5 El cliente abre la conexión **a través del gateway** incluyendo el ID token: io(url, { auth: { token }, query: { token } }) — el ?token= en la URL del upgrade es obligatorio para que el gateway pueda validarlo (limitación documentada de Socket.IO v4: auth.token llega después del upgrade HTTP; ver GATEWAY_RESUMEN.md §6 TODO(ws-auth)).

- 6 WsProxyService valida el token con Firebase, enriquece la identidad, **rechaza** con 401/403 si falta token o el usuario no está ACTIVE, **sustituye** ?userId= por el userId verificado (cierra el hueco histórico de notifications, que aceptaba cualquier userId) y **elimina** ?token= antes de reenviar el upgrade al servicio.
- 7 En orders, el gateway WS del chat vuelve a autenticar el handshake (auth.token → GET /auth/validate de identity) salvo en modo AUTH_DISABLED (despliegue detrás del gateway), donde confía en el userId inyectado.

15.3 Contrato de eventos del chat (orders)

Evento	Dirección Semántica	
conversation:joined / conversation:left	C→S	Entrar/salir de la sala de una conversación
message:new	C→S y S→C	Nuevo mensaje (también persiste y publica order.chat.message.sent al bus)
message:read	C→S y S→C	Confirmación de lectura
typing:start / typing:stop	C→S y S→C	Indicador de escritura
order:status-updated	S→C	El estado del pedido cambió (bridge de la saga al cliente vía RealtimeHubService)
session:revoked (interno)	bus→S	identity.user.deactivated → desconexión forzada de todos los sockets del usuario
error	S→C	Autenticación inválida antes de desconectar

Salas: user:<userId> (lista de chats y avisos personales) y una sala por conversación. El historial es consultable por REST (/conversations, /messages), de modo que el WS es una optimización, no la fuente de verdad.

15.4 Contrato del canal de notificaciones

- Sala por usuario (room(userId)); múltiples conexiones simultáneas soportadas (multi-pestaña).
- Evento único **notification** con { title, body, data } (más imageUrl cuando aplica).
- La entrega en vivo es *best effort*: la notificación **siempre** queda persistida en la bandeja y el contador de no-leídas; isUserOnline informa presencia interna para el canal REALTIME.
- El cliente usa socket.io-client (dependencia del frontend) apuntando al gateway con ?token=.

15.5 Consideraciones de escala

El estado de salas vive en memoria de cada proceso (no hay adapter Redis de Socket.IO en las dependencias). Con una réplica por servicio esto es correcto; con N réplicas los broadcasts se fragmentarían — restricción registrada en Riesgos (R-06). *Información pendiente de confirmar con el equipo de desarrollo: número de réplicas configurado en producción para orders/notifications.*

16. Diagrama de interacción entre servicios

[Insertar aquí el diagrama]

Explicación del diagrama

El mapa de interacción condensa **quién habla con quién y por qué medio**. Su lectura clave es la asimetría entre los dos planos de comunicación:

Plano síncrono (flechas sólidas). Es deliberadamente escaso y en forma de estrella alrededor del gateway. Fuera del proxy, solo existen cinco llamadas REST internas de negocio (orders→financial, orders→identity, orders→products, products→identity, financial→Wompi) — todas son *preguntas* cuya respuesta se necesita en línea, y tres de ellas tienen degradación explícita para no acoplar disponibilidad. No hay cadenas síncronas de más de un salto: el gateway nunca es atravesado por tráfico este-oeste.

Plano asíncrono (flechas punteadas al bus). Todos los *hechos* de negocio pasan por el topic `eciexpress_events`. El diagrama muestra a orders como el mayor productor/consumidor (es el coordinador de la saga), a notifications como consumidor universal (se suscribe a todos los dominios con comodines), a identity como productor puro y a reporting como el único servicio completamente fuera del bus (observa por métricas). Las etiquetas de las flechas son los routing keys reales del código, lo que convierte este diagrama en un índice visual del catálogo de la sección 17.

Plano de tiempo real (flechas onduladas). Dos túneles WebSocket atraviesan el gateway (chat y notificaciones), autenticados en el upgrade.

El lector debe concluir: (a) el sistema tolera la caída de cualquier consumidor sin perder hechos (bus + outbox), (b) las dependencias síncronas están minimizadas y protegidas, y (c) el acoplamiento real del sistema está en los **contratos de eventos**, no en llamadas directas.

17. Catálogo de eventos

Catálogo consolidado y **verificado contra el código** (archivos de contratos: orders-service/.../event-contracts.ts, products-service/.../contracts.ts y messaging/events/*.ts, financial-service/.../event-patterns.ts, notifications-service/.../event-patterns.ts, handlers de fulfillment, catálogo de identity). Todos los eventos llevan además el sobre estándar (§14.2). Montos en centavos COP.

17.1 Dominio identity (productor: identity-service)

Evento (routing key)	Publicador	Consumidores	Payload (campos de negocio)	Descripción
identity.user.registered	identity	notifications, financial	userId, firebaseUid, email, fullName, systemRole	Primer login / sync-profile: crea contacto (notifications) y billetera (financial)
identity.user.profile_updated	identity	notifications	userId, changedFields[], newValues	Actualiza email/teléfono guardados (habilita WhatsApp/SMS)
identity.user.deactivated	identity	orders, financial, notifications	userId, reason	Revoca sockets del chat (orders), desactiva billetera/contacto
identity.user.role_changed	identity	notifications	userId, roleId, roleName, action, performedBy	Asignación/revocación de rol
identity.role.updated	identity	— (informativo)	roleId, permissionIds[], performedBy	Permisos de un rol reemplazados
identity.store.created	identity	products, financial, fulfillment, notifications	storeId, ownerId, name, type, location, isActive	Crea réplicas/proyecciones y el mapeo tienda→dueño
identity.store.updated	identity	products (reconcilia),	storeId, changedField snapshot en products)	Notificación de cambio (pull del snapshot en products)

		financial, notifications	s[], performedBy	
identity.store.status_changed	identity	orders, products, financial, notifications	storeId, previousStatus, newStatus, reason, performedBy ?	OPEN/CLOSED/TEMPORARILY_CLOSED; orders bloquea pedidos si cierra
identity.store.schedule_changed	identity	products (reconcilia), notifications	storeId, scheduleId, action, dayOfWeek, performedBy	Horario semanal upsert/update/delete
identity.store.temporarily_closed	identity	notifications, products	storeId, closureId, startDate, endDate, reason, performedBy	Cierre temporal programado
identity.store.closure_cancelled	identity	notifications, products	storeId, closureId, performedBy	Cancelación de cierre
identity.store.closure_expired	identity	products, notifications	storeId, closureId	Job automático al vencer el cierre (performedBy = null)
identity.store.staff_changed	identity	fulfillment, notifications	storeId, userId, action, performedBy	Alimenta la proyección de autorización vendedor ↔ tienda

17.2 Dominio order (productor: orders-service)

Evento	Publicador	Consumidores	Payload	Descripción
order.cart.created	orders	products	cartId, buyerId, storeId, currency	Nace el carrito (orden DRAFT); products crea la proyección (cartId == orderId)
order.cart.item_changed	orders	products	cartId, storeId, currency, items[{productId, quantity}]	Cambio de líneas → products recotiza y responde products.cart.priced

order.order.created	orders	financial, products, notifications	orderId, buyerId, storeId, orderAmount, peakFeeAmount, isPeakHour, totalAmount, paymentMethod	Checkout: dispara retención del cobro y reserva de stock
order.order.status_changed	orders	notifications	orderId, buyerId, status	Cada transición de la máquina de estados
order.order.confirmed	orders	fulfillment, products, notifications	orderId, orderNumber, buyerId, storeId, pickupExpiresAt	Pedido confirmado: fulfillment proyecta; products consume la reserva
order.order.ready_for_pickup	orders	fulfillment, notifications	orderId, buyerId, storeId, ...	Pedido listo: fulfillment genera el código de retiro
order.order.cancelled	orders	financial, fulfillment, products, notifications	orderId, buyerId, wasSold, refundPolicy?	Cancelación con política de reembolso e instrucción de stock
order.chat.message.sent	orders	notifications	messageId, conversationId, senderId, recipientId, preview	Push del chat al receptor
order.return.requested	orders	products	orderId, storeId, full, items?, reason?	Solicitud de devolución → products cotiza
order.return.confirmed	orders	financial	orderId, buyerId, storeId, full, refundAmount	Devolución autorizada → financal acredita

17.3 Dominio products (productor: products-service)

Evento	Publicad Consumidores or		Payload	Descripción
products.cart.priced	products	orders	cartId, storeId, currency, lines[{{productId, name, imageUrl, listUnitPrice, unitPrice, quantity, promos)	Cotización autoritativa del carrito (precios + promos)

			totalAmount, appliedPromotionId?}}, subtotalAmount, discountAmount, finalAmount	
products.return.priced	products	orders	orderId, storeId, full, refundAmount, lines[]	Monto de la devolución calculado
product.inventory.reservation_confirmed	products	orders	orderId, storeId	Stock reservado para TODAS las líneas — gate de confirmación
product.inventory.reservation_rejected	products	orders	orderId, productId, storeId, requestedQuantity, availableQuantity, reason	Sin stock para una línea
product.inventory.low_stock	products	notifications	productId, storeId, name, stock, reservedStock, minStock	Alerta al dueño de la tienda
product.item.created/updated/deleted	products	— (informativos/reporting futuro)	datos del producto	Ciclo de vida del catálogo
product.category.created/updated/deleted	products	—	datos de categoría	Ciclo de vida de categorías
product.promotion.created/updated/deleted	products	—	datos de promoción	Ciclo de vida de promociones

17.4 Dominio financiero (productor: financial-service)

Evento	Publicador	Consumidores	Payload	Descripción
financial.payment.processed	financial	orders, notifications	orderId, (userId, totalCharged esperados por notifications)	Cobro retenido con éxito (HELD)
financial.payment.failed	financial	orders, notifications	orderId, reason?	Falla de cobro (saldo insuficiente, tienda inválida...) → pedido FAILED
financial.payment.released	financial	notifications	orderId, storeId, storePayoutAmount?	Retención liberada al negocio tras la entrega
financial.refund.issued	financial	notifications	orderId, refundedAmount?, (userId esperado)	Reembolso acreditado
financial.wallet.topup.approved	financial	notifications	topupId, amount, (userId esperado)	Recarga aprobada por Wompi
financial.wallet.topup.failed	financial	notifications	topupId, reason?	Recarga rechazada

Advertencia de contrato vigente: los eventos de financial históricamente publicaban walletId/storeId sin el userId del comprador; sin userId, notifications omite la notificación. Ver Deuda Técnica DT-04.

17.5 Dominio fulfillment (productor: fulfillment-service)

Evento	Publicador	Consumidores	Payload	Descripción
fulfillment.qr.generated	fulfillment	notifications	orderId, buyerId, qrCode (URL PNG), shortCode, expiresAt	Código de retiro creado (correo + WhatsApp con imagen)
fulfillment.qr.expiring_soon	fulfillment	notifications	orderId, buyerId, ...	Aviso previo al vencimiento
fulfillment.qr.expired	fulfillment	orders, notifications	orderId, buyerId	Código vencido → orders cancela con NO_REFUND

fulfillment.delivery.confirmed	fulfillment	orders, financial, notifications	orderId, buyerId	Entrega confirmada → DELIVERED + liberación del pago
fulfillment.delivery.failed	fulfillment	orders, notifications	orderId, buyerId, reason?	Entrega fallida → pedido FAILED

17.6 Canal de comandos

Evento	Publicador	Consumidor	Payload	Descripción
notification.send.requested	cualquier servicio	notifications	recipientUserId y/o email/phone, channels[], title, body, type?, data?, dedupKey?	Comando genérico para notificaciones a la medida (equivalente al REST interno POST /notifications/send)

18. Vista de datos

18.1 Estrategia de persistencia

Decisión	Implementación	Justificación
Database-per-service	6 bases PostgreSQL independientes en Neon (identity, products, orders, financial, fulfillment, notifications); reporting sin BD	Autonomía de esquemas y despliegues; imposibilita el acoplamiento por datos
ORM por equipo, misma familia SQL	Prisma 7 en identity; TypeORM en products/orders/financial/fulfillment/notifications	Libertad táctica dentro de una restricción estratégica (PostgreSQL único)
Migraciones versionadas	prisma/migrations/* (identity), src/migrations / database/migrations (TypeORM) con DB_SYNCHRONIZE=false	Evolución de esquema reproducible y auditable
Dinero como enteros	bigint con transformer (financial), int centavos (orders/products)	Elimina errores de coma flotante; convención transversal COP
Proyecciones locales	tiendas (products/financial/fulfillment/notifications), carrito (products), pedido y staff (fulfillment), contactos (notifications)	Autonomía en el hot path: decidir sin llamadas síncronas

Snapshot en el pedido	order_items congela nombre, precio e imagen al momento de la compra	El historial no cambia si el catálogo cambia
Tablas de mensajería	outbox_events (identity/orders/fulfillment), processed_events (orders/fulfillment; dedupe equivalente en notifications)	Publicación confiable + consumo idempotente
Auditoría append-only	audit_logs (identity, fulfillment), order_status_history (orders)	Trazabilidad de acciones sensibles con actor y razón
Objetos binarios fuera de la BD	Azure Blob Storage: products (imágenes+GLB), store-logs/store-banners, avatares, orders (evidencia devoluciones <orderId>/refunds/<refundId>/), recibos PDF	Las BDs guardan URLs/paths, nunca binarios
Caché	Redis (gateway: enriquecimiento TTL 60 s + rate limit; identity: REDIS_URL disponible); in-memory (permisos en identity)	Estado efímero recomputable, con degradación
Consistencia entre servicios	Eventual, por eventos + reconciliación pull	Coherente con la saga coreografiada

18.2 Dueños de datos por concepto de negocio

Concepto	Dueño (escritura)	Réplicas/proyecciones (lectura)
Usuario, roles, permisos, sesiones	identity	notifications (contactos), gateway (caché efímera)
Tienda (existencia, estado, horarios, staff)	identity	products, financial (+config financiera propia), fulfillment (staff), notifications (owner), orders (directorío para bloqueo)
Producto, precio, promoción, stock	products	orders (snapshot en ítems), frontend (lectura)
Carrito	orders (estado DRAFT)	products (proyección para cotizar/reservar)
Pedido y su estado	orders	fulfillment (proyección operativa)
Dinero (saldos, transacciones, comisiones)	financial	—
Código de retiro y entregas	fulfillment	—
Notificaciones y preferencias	notifications	—

19. Diagramas de bases de datos

Un diagrama por base de datos. Los esquemas provienen de schema.prisma (identity) y de las entidades TypeORM/migraciones (resto).

19.1 identity_db

[Insertar aquí el diagrama]

El modelo tiene tres zonas nítidas. La zona **Identidad/RBAC** implementa el control de acceso en su forma normalizada clásica: users (sin contraseñas — el vínculo con Firebase es firebaseUid único) se relaciona con roles a través de user_roles (con vigencia opcional expiresAt y trazabilidad de quién asignó), y los roles agregan permissions granulares (recurso:acción) vía role_permissions. user_sessions materializa la política de sesión única: un sessionId UUID activo por usuario con expiración a 8 horas.

La zona **Administración** modela el punto de venta: stores pertenece a un dueño y agrega store_staff (vendedores autorizados, con historial de remoción), store_schedules (un registro por día de semana, restricción única que impide duplicados) y store_closures (cierres programados con ciclo de vida propio SCHEDULED→ACTIVE→EXPIRED/CANCELLED, procesados por el job del scheduler).

La zona **Transversal** contiene la fontanería arquitectónica: outbox_events (la cola saliente transaccional, con idempotencyKey única, contadores de reintento y nextRetryAt para el backoff), audit_logs (bitácora append-only con actor, objetivo, acción tipificada y valores antes/después) y el módulo PQRS (tickets con hilo de mensajes tipificado por rol del emisor).

19.2 products_db

[Insertar aquí el diagrama]

El centro del modelo es products, cuya entidad TypeORM está íntegramente verificada: además de los datos comerciales (nombre, slug por tienda, precio decimal(10,2), SKU), incorpora **tres contadores de inventario** que sostienen la saga de reserva — stock (físico), reserved_stock (comprometido por pedidos en curso) y min_stock (umbral de la alerta low_stock; el disponible real es stock - reserved_stock) — y **el pipeline 3D** como máquina de estados embebida (model_generation_status/progress/error +

model_3d_url), con las cuatro URLs de imagen (principal, frontal, lateral, trasera) que alimentan la generación multivista.

categories forma un árbol (endpoint /categories/tree) y restringe el borrado si tiene productos (ON DELETE RESTRICT). promotions define descuentos con objetivo y vigencia, cuya aplicación se refleja en el effective-price y en el campo appliedPromotionId de la cotización del carrito. inventory_movements registra el historial consultable por producto/tienda. En la zona de proyecciones, el carrito replica lo mínimo para cotizar y reservar (con la invariante cart_id == orderId), y stores_replica guarda el snapshot reconciliado desde identity. Los nombres exactos de las tablas de carrito/réplica/auditoría siguen a sus entidades TypeORM; su semántica es la descrita. *Información pendiente de confirmar con el equipo de desarrollo: nomenclatura final de esas tablas auxiliares en producción.*

19.3 orders_db

[Insertar aquí el diagrama]

orders es la tabla más rica del sistema porque un pedido concentra el estado de la saga completa. Además de las referencias lógicas a comprador y tienda (UUIDs de identity, sin FK física — frontera de servicio), persiste: el **estado** entre los 13 posibles, el flag stock_reserved (la mitad del gate de confirmación digital), el idempotency_key del request de creación (anti-duplicados), la ventana de retiro (pickup_expires_at, insumo de la expiración del QR), el snapshot monetario en centavos, y el bloque de campos de **devolución pendiente** (pending_return_* guarda la cotización mientras un ADMIN decide, incluyendo el estado del que vino para restaurarlo si se rechaza; last_return_reason/refund_id conservan el motivo y la carpeta de evidencia en Blob).

order_items congela el catálogo al momento de la compra (nombre, precio, imagen, notas del comprador tipo “sin cebolla”). order_status_history da la trazabilidad de cada transición con su **actor tipificado** — la evidencia de quién movió el pedido: cliente, vendedor, sistema, pago o fulfillment. order_ratings es 1:1 con el pedido.

La zona de comunicación modela el chat: conversations (una por pedido, con campos de presentación y archivado), participants con rol, y messages con estado de lectura. La zona de mensajería contiene el outbox del productor y processed_events del consumidor idempotente — las dos tablas que hacen confiable la coreografía.

19.4 financial_db

[Insertar aquí el diagrama]

El modelo financiero gira alrededor de dos agregados. **wallets** mantiene el saldo como bigint en centavos (transformer dedicado — jamás flotantes) con relación 1:1 al

usuario; **wallet_topups** registra cada recarga con su método, su estado, la respuesta cruda de Wompi en jsonb (evidencia completa para disputas), el id de transacción del proveedor y la ruta del recibo PDF en Blob.

order_transactions es la implementación del *escrow*: una fila por pedido (la restricción **única sobre order_id** es simultáneamente regla de negocio y mecanismo de idempotencia frente a eventos duplicados) con el desglose completo del dinero — **order_amount** (productos), **peak_fee_amount** (recargo fijado por orders), **total_charged** (lo debitado), **platform_fee_amount** (comisión calculada con el porcentaje de la tienda) y **store_payout_amount** (neto del negocio) — más el ciclo temporal **held_at** → **released_at** / **refunded_at** y **refunded_amount** para reembolsos parciales. El estado (HELD, RELEASED, REFUNDED/parciales, FAILED con **failure_reason** tipificado: tienda inexistente/inactiva, billetera inexistente, fondos insuficientes) narra la vida del dinero de cada venta.

La tabla **stores** es una proyección enriquecida: a los datos replicados de **identity** les suma la **configuración financiera propia** de este dominio — porcentaje de comisión, franja y recargo de hora pico (hora inicio/fin + días), y la cuenta de payout del negocio.

19.5 fulfillment_db

[Insertar aquí el diagrama]

El diagrama refleja la decisión central de fulfillment: la credencial de retiro es **estado en base de datos, no un JWT** — “el estado vive en la base porque el código es de un solo uso, invalidable y regenerable”. **pickup_codes** guarda el token opaco (lo que codifica el QR y nombra el PNG público-inadivinable) junto con su representación humana (**short_code** base32 sin caracteres ambiguos) y su ciclo de vida: **ACTIVE** (única activa por pedido — restricción reforzada por advisory locks ante eventos concurrentes), **USED** al confirmar, **EXPIRED** por el job de expiración e **INVALIDATED** por cancelación.

deliveries registra el desenlace operativo (entrega por QR o manual, quién la confirmó, fallos con motivo). Las dos **proyecciones** le dan autonomía: **order_projection** (alimentada por **order.order.confirmed**) evita consultar a **orders** para validar un retiro, y **store_staff_projection** (alimentada por **identity.store.created/staff_changed**) resuelve la autorización vendedor↔tienda sin llamar a **identity**. La zona infra repite el patrón **outbox** + **processed_events** + auditoría.

19.6 notifications_db

[Insertar aquí el diagrama]

El modelo separa **el hecho** (la notificación, siempre persistida en la bandeja con título, cuerpo, data para deep-links y **image_url** cuando el mensaje lleva imagen — p. ej. el

QR) de **sus entregas** (notification_deliveries: una fila por canal intentado, con estado SENT/FAILED/SKIPPED y respuesta del proveedor; el scheduler de reintentos opera sobre las fallidas). Esta separación permite que “el usuario desactivó WhatsApp” o “Twilio falló” no afecten la bandeja in-app ni los demás canales.

recipients es la libreta de contactos local construida por eventos de identity (el email llega con el registro; el teléfono solo si el perfil lo aporta — condición para WhatsApp/SMS), e incluye el mapeo tienda→dueño que permite “notificar a la tienda”. notification_preferences guarda el opt-out por canal (todo activo por defecto). La tabla de deduplicación completa el consumo idempotente. La migración RemovePush es la evidencia en esquema de que el canal Push/FCM fue retirado del alcance.

19.7 Nota sobre reporting

reporting-service no tiene base de datos propia (ver §12.7); consulta Application Insights y, en solo lectura, orders_db y financial_db. No aplica diagrama.

20. Diagramas de secuencia relevantes

Convención para todos los diagramas: los participantes “Bus” representan el topic eciexpress_events de Azure Service Bus; toda publicación pasa por el outbox del productor aunque no se dibuje cada vez (se dibuja explícito solo donde es esencial). Las flechas punteadas son eventos asíncronos.

20.1 Registro e inicio de sesión

[Insertar aquí el diagrama]

La autenticación tiene dos tiempos. Primero, el **login es contra Firebase**: la SPA obtiene un ID token JWT firmado por Google; el backend jamás ve credenciales. Segundo, la **sincronización de perfil** (POST /auth/sync-profile) crea el usuario local en el primer login — la ruta es “semi-pública” en el gateway: se verifica el token pero se omite el enriquecimiento, precisamente porque el perfil local aún no existe. La creación del usuario y la escritura del evento UserRegistered en el outbox ocurren en la misma transacción; el worker lo publica y notifications crea el contacto y envía la bienvenida. La llamada es idempotente (200 si ya existía), y el servicio crea la **sesión única** de 8 horas cuyo sessionId viajará como X-Session-Id. El diagrama cierra mostrando el régimen permanente (enriquecimiento con caché de 60 s) y el caso de error distintivo del onboarding: 403 PROFILE_NOT_FOUND, que el frontend usa para redirigir al flujo de registro.

20.2 Creación de pedido (carrito → checkout)

[Insertar aquí el diagrama]

El carrito **es** una orden en estado DRAFT, y su cotización es un ida-y-vuelta asíncrono con products: orders publica el contenido (cart.item_changed) y products responde con la **cotización autoritativa** (cart.priced) que incluye promociones aplicadas y totales — orders nunca calcula precios de catálogo. En el checkout ocurre la única llamada síncrona del flujo: orders consulta a financial el recargo de hora pico y lo **congela** en el total (regla “precio visto = precio cobrado”); la llamada está protegida por timeout y degradación (recargo 0) para no bloquear ventas. order.order.created sale con el desglose completo y dispara, en paralelo, el cobro (financiamiento) y la reserva de stock (products) — continuación en 20.3 y 20.4. La bifurcación final refleja la “Option C”: efectivo pasa a CREATED (espera solo la reserva), digital a PENDING_PAYMENT (espera pago **y** reserva).

20.3 Validación y reserva de inventario

[Insertar aquí el diagrama]

La reserva implementa el patrón **reservar-luego-consumir** en dos fases sobre los contadores stock/reserved_stock. Al llegar order.order.created, products toma las cantidades de **su proyección** del carrito (defensa contra payloads incompletos) y reserva línea por línea bajo bloqueo; el resultado es todo-o-nada: reservation_confirmed solo si **todas** las líneas quedaron comprometidas, o reservation_rejected con el disponible real de la línea fallida. Ese evento cierra el gate de confirmación en orders (§14.5) — el comentario del contrato lo explica: “evita que dos compradores por la última unidad vean ambos ‘confirmado’”. La reserva se **consume** (descuenta stock físico) recién con order.order.confirmed, y el flag wasSold de las cancelaciones decide la compensación correcta: liberar una reserva viva o restituir stock ya vendido. El umbral min_stock dispara la alerta al vendedor por el canal de notificaciones.

20.4 Pago aprobado

[Insertar aquí el diagrama]

El “pago” del MVP es un **débito de billetera con retención**: no hay cargo a tarjeta en el checkout (las tarjetas participan en las *recargas* vía Wompi, §20.11-nota). El manejador es idempotente por partida doble (consulta previa + restricción única con la violación 23505 tratada como duplicado benigno), valida las precondiciones de negocio y ejecuta el débito y la creación de la transacción HELD en **una sola transacción de base de datos** — o ambos o ninguno. El desglose respeta la separación de autoridades: orders fijó el recargo de hora pico; financial calcula lo suyo (comisión de plataforma y neto del negocio). financial.payment.processed empuja la

saga: orders aprueba el pago y, si el stock ya está reservado, confirma — encadenando la proyección en fulfillment y las notificaciones multicanal. El dinero queda retenido hasta la entrega (20.7) o una compensación (20.8).

20.5 Pago rechazado

[Insertar aquí el diagrama]

El camino triste del cobro muestra la **compensación** de la saga. financial nunca lanza: registra la transacción FAILED con la causa tipificada y publica payment.failed con reason legible (que queda en el historial del pedido — el catálogo de orders pide explícitamente incluirla). orders transita a FAILED con actor payment, y la definición de dominio STOCK_RELEASE_STATUS documenta que, para el inventario, CANCELLED y FAILED significan lo mismo: liberar cualquier reserva viva. El comprador se entera por dos vías coherentes (notificación push/mail y el evento WS order:status-updated del propio chat del pedido). No hay reintento automático del cobro: el modelo es crear un pedido nuevo tras recargar saldo — decisión coherente con la idempotencia por order_id único en financial.

20.6 Generación del QR

[Insertar aquí el diagrama]

El disparador del QR **no** es la confirmación del pedido sino ready_for_pickup — el código existe cuando hay algo físico que recoger, y su vencimiento hereda la ventana pickupExpiresAt fijada por orders. La secuencia exhibe las tres defensas de idempotencia de fulfillment en orden: advisory lock transaccional (serializa el mismo evento entregado dos veces en paralelo — el bug documentado de los QR duplicados), chequeo de processed_events, y unicidad de código ACTIVE por pedido. La credencial nace en dos formas equivalentes (token para el QR, código corto para teclear) y viaja por dos canales: la notificación (el correo y WhatsApp insertan la **URL pública-inadivivable** del PNG, servida sin token porque los clientes de correo/Meta no pueden autenticarse) y la consulta REST del comprador (protegida: solo el dueño), donde la SPA renderiza el QR localmente.

20.7 Confirmación de entrega

[Insertar aquí el diagrama]

Es el momento donde **la custodia del dinero cambia de manos**, y por eso cada paso está blindado: la validación comprueba tres cosas (código activo, vigencia, y autorización del vendedor contra la proyección local de staff — sin llamar a identity), y la confirmación escribe el uso del código, el registro de entrega y el evento **en la misma transacción** con outbox ("garantiza que delivery.confirmed (que mueve dinero)

nunca se pierda”). Aguas abajo el evento tiene dos consumidores críticos: orders cierra el ciclo del pedido (DELIVERED, chat clausurado) y financial ejecuta la segunda mitad del escrow (HELD → RELEASED, acreditando el neto — valor de productos menos comisión — al negocio). La entrega manual comparte todo el camino posterior; solo cambia el origen de la confirmación.

20.8 Cancelación y reembolso

[Insertar aquí el diagrama]

La cancelación es la **transacción compensatoria** de la saga y el diagrama muestra su matiz más fino: el evento lleva **instrucciones, no solo el hecho**. refundPolicy le dice a financial cuánto devolver según la equidad del caso — cancelación del cliente antes de que el pedido esté listo: 50% de los productos (el negocio ya trabajó) y pierde el recargo de hora pico; expiración del QR: 0% (el pedido quedó preparado y sin recoger, el negocio cobra completo); ausencia de política: 100% (cancelaciones del sistema, p. ej. reserva rechazada). Y wasSold le dice a products qué compensación de inventario aplicar: si el pedido había llegado a CONFIRMED o más allá, el stock físico ya se descontó y hay que **restituirlo**; si no, basta con soltar la reserva. fulfillment participa invalidando la credencial. El guard que impide al cliente cancelar en READY_FOR_PICKUP — aunque la máquina de estados permita esa transición para fulfillment — es la regla que hace posible la política NO_REFUND.

20.9 Notificación de cambio de estado

[Insertar aquí el diagrama]

Cada transición del pedido produce **dos** flujos en vivo complementarios y desacoplados: el del dominio (orders → su propio WebSocket → la vista del pedido abierta, evento order:status-updated) y el de notificaciones (bus → catálogo → canales). El diagrama detalla el segundo: la deduplicación de entrada, la resolución por catálogo (qué estados notifican, a quién y por qué canales), la **persistencia incondicional** en la bandeja (la campana funciona aunque el usuario esté offline), el fan-out por canal con registro individual del resultado (notification_deliveries), y el fallback plantilla-HTML→texto-plano del correo. La separación bandeja/entregas explica la resiliencia del subsistema: el fallo de un proveedor externo jamás pierde la notificación in-app.

20.10 Chat entre comprador y vendedor

[Insertar aquí el diagrama]

El chat es **por pedido** (una conversación por orden, con participantes tipificados) y su autenticación es doble por diseño: el gateway valida el token en el upgrade (y neutraliza suplantaciones reescribiendo el userId), y el gateway WS de orders vuelve a

validar el handshake contra identity — defensa en profundidad para el único canal de larga vida del sistema. Los mensajes tienen doble vía de entrada (WS o REST) y triple salida: persistencia (historial consultable), broadcast a la sala (tiempo real para quien está en la conversación) y evento al bus (para que notifications alcance al receptor ausente con el preview). Los indicadores de escritura son deliberadamente efímeros. El paso final conecta el chat con el ciclo de identidad: la desactivación de un usuario derriba sus sockets en caliente vía el hub interno — y con el ciclo del pedido: la entrega cierra el chat, y solo una devolución post-entrega lo reabre.

21. Arquitectura general de seguridad

21.1 Autenticación

Mecanismo	Implementación	Evidencia
Identidad federada	Firebase Auth (Google / email+password). El backend nunca almacena credenciales; identity guarda el perfil de negocio vinculado por firebaseUid	identity-service/README.md ("regla de oro"), firebase-admin en gateway e identity
Verificación de tokens	verifyIdToken del Firebase Admin SDK en el gateway (todas las rutas protegidas) y en identity (autoridad final)	gateway/src/firebase/firebase.service.ts, identity/src/common/guards/firebase-auth.guard.ts
Sesión única	X-Session-Id (UUID) emitido en el sync-profile, validado por identity; TTL 8 horas; tabla user_sessions	session.service.ts (SESSION_TTL_MS = 8h), frontend api.ts envía el header desde sessionStorage
WebSockets	Token obligatorio en el upgrade (?token=); el gateway valida, enriquece y reescribe el userId; orders re-valida el handshake contra identity	ws-proxy.service.ts, communication.gateway.ts
Webhook Wompi	Sin JWT: autenticación por checksum SHA-256 sobre el body crudo con el secret de eventos; el gateway reenvía el buffer sin transformar (rawBody: true)	webhooks.controller.ts, wompi.service.ts, GATEWAY_RESUMEN.md §9

21.2 Autorización

Nivel	Mecanismo	Detalle
Perimetral	Política de rutas del gateway	Bloqueadas (404): /identity/internal/*, /notifications/send, /notifications/wakeup, Swagger /api de todos. Públicas: health, catálogo de tiendas de solo lectura (lista precisa que excluye /stores/my y /stores/user/:id), GET model3d, QR/imágenes de fulfillment, webhook Wompi. Estado de cuenta: usuarios no ACTIVE → 403 USER_INACTIVE sin tocar el downstream
RBAC	Roles de sistema BUYER < VENDOR < ADMIN (+ANALYST) con effectiveRole por jerarquía; roles personalizados posibles (isSystem=false)	Seed de permisos por rol (user:read/write, role:*, store:read/write/close/staff, audit:read); PermissionsGuard + @RequirePermission('recurso:accion') en identity
Por servicio	Guards de confianza en headers	orders: AdminGuard, decoradores de usuario actual; products: x-user-role/x-user-store; fulfillment: pertenencia vendedor→tienda contra store_staff_projection; reporting: RolesGuard (ADMIN/ANALYST); financiero: AdminGuard y StoreStaffGuard (el gateway anota que el AdminGuard de financiero "es placeholder (TODO)" — ver DT-05)
Por recurso	Reglas de propiedad	El comprador solo ve su código de retiro (buyerId == x-user-id); dueño/ADMIN para editar tienda; participante para entrar a una conversación

21.3 Anti-spoofing y saneamiento

- **Strip de headers:** el gateway **elimina** x-user-id, x-user-role, x-user-store, x-store-id provenientes del cliente antes de inyectar los verificados ("seguridad: previene header spoofing").
- **Validación de entrada:** ValidationPipe global con whitelist: true y forbidNonWhitelisted: true (orders y servicios TypeORM); Zod pipes en identity.
- **SQL:** ORMs parametrizados; en reporting, catálogo cerrado de consultas con binds obligatorios y statement_timeout ("NUNCA se interpola entrada del usuario en el SQL").
- **Rate limiting:** 60 req/60 s por uid o IP, distribuido en Redis (clave uid: solo para conteo; la autorización real sigue siendo del token).

21.4 Gestión de secretos y cifrado

Aspecto	Práctica observada
Secretos de runtime	Variables de entorno / secrets de Container Apps (service account de Firebase como JSON o archivo; connection strings; claves de Wompi con getOrThrow) — nunca commiteados (.env en .gitignore, .env.example como plantilla)
CI/CD	OIDC de GitHub → Azure ("login OIDC al Service Principal, sin secretos de larga vida"); token de SWA como secret
Mensajería en producción	Managed Identity (DefaultAzureCredential + FQNS) — sin connection strings en producción
Tránsito	HTTPS/WSS terminado por Azure (SWA/Container Apps); TLS a Postgres (sslmode=require Neon) y a Redis (rediss:// en prod); ACME HTTP-01 para certificado del dominio propio
Datos de pago	Solo tokens de Wompi; "NUNCA datos crudos de tarjeta" (wompi.service.ts)
Frontend	Únicamente claves públicas por diseño (Firebase web config, Wompi public key) — documentado en el workflow
Blob	Contenedores de imágenes con acceso público de solo blob (URLs); la AccountKey nunca llega al frontend; recibos accedidos vía URL emitida por el backend

21.5 CORS, filtros e interceptores

- **CORS:** lista blanca de orígenes en el gateway (CORS_ORIGINS); los WS declaran CORS propio detrás del proxy.
- **Filtros de excepción** globales por servicio (AllExceptionsFilter, HttpExceptionHandler) → formato de error homogéneo sin fugas de stack.
- **Interceptores:** CorrelationIdInterceptor (identity) y generación/propagación de x-correlation-id en el gateway; middlewares de logging con contexto por request.
- **Auditoría:** audit_logs (identity, fulfillment) y order_status_history con actor — trazabilidad de acciones sensibles.

21.6 Seguridad entre servicios

El modelo interno es **confianza por red + identidad por headers**: los endpoints internal/* están marcados @Public() en identity pero son inaccesibles desde internet (bloqueo en gateway + red interna de Container Apps); los servicios aceptan x-user-id porque solo el gateway puede alcanzarlos. Este modelo es explícito en la documentación del compose ("aproximando el aislamiento de red que tendrán en

Azure Container Apps”) y su supuesto de red es el punto a vigilar (Riesgo R-02). No hay mTLS ni tokens service-to-service (*Información pendiente de confirmar con el equipo de desarrollo: reglas de ingress internas exactas configuradas en ACA*).

21.7 Buenas prácticas adicionales verificadas

- Contraseña: cambio delegado en Firebase (POST /auth/change-password en identity).
- Códigos QR: tokens opacos de alta entropía, un solo uso, invalidables, con expiración — no JWT.
- Notificaciones: POST /notifications/send interno bloqueado del público; el WS de notifications ya no acepta userId sin verificar (reescrito por el gateway).
- Swagger no expuesto en producción a través del gateway.
- Idempotencia de mutaciones sensibles (header Idempotency-Key en orders).

22. Arquitectura general de observabilidad

22.1 Telemetría centralizada

Azure Application Insights es el backbone: los 8 servicios incluyen el SDK applicationinsights inicializado **antes** de crear la app NestJS (“para que el SDK pueda instrumentar HTTP/dependencias. No-op si no hay connection string (local)” — app-insights.ts presente en cada servicio con su service-name.ts). Esto produce por servicio: requests, dependencias (HTTP, Postgres, Service Bus), excepciones y trazas correlacionadas.

22.2 Logs

Servicio	Logger	Formato
identity	pino + pino-http	JSON estructurado
fulfillment	nestjs-pino + pino-http	JSON estructurado
products	winston + nest-winston (+ transports propios)	JSON estructurado

orders, financial, notifications, reporting, gateway	StructuredLogger propio + middleware de logging con logging.context	JSON con contexto de request
--	---	------------------------------

Convención transversal: contexto de correlación (x-correlation-id propagado por el gateway y presente en el sobre de eventos) — permite unir la traza HTTP con la traza de eventos de una misma operación.

22.3 Métricas y KPIs

- **Técnicos (reporting /kpis):** overview (volumen, tasa de error, p95, top excepciones; ventana acotada 1 h–30 d), latencia por servicio/minuto, **flujo de eventos entre servicios** (generados/recibidos/fallidos — la herramienta para detectar sagas atascadas), logs por servicio.
- **De negocio:** quick queries allowlisted sobre orders_db/financial_db (read-only, con timeout).
- **De infraestructura:** métricas del namespace de Service Bus (Azure Monitor Metrics) y estado de despliegues (GitHub Actions + revisiones de Container Apps vía ARM).

22.4 Health checks

Servicio	Endpoint(s)	Detalle
Todos	GET /health	Liveness básico (público vía gateway)
products	GET /health/live, GET /health/ready, GET /health	@nestjs/terminus con indicador de Service Bus (service-bus.health.ts)
products	GET /ai-health	Salud del pipeline 3D (servicio de IA alcanzable)
gateway	GET /health	Propio, sin auth
reporting	GET /kpis/health	Agregador: consulta la salud de los 7 servicios
Compose healthchecks local	Compose healthchecks de Redis y SQL Edge	Orquestación de arranque

22.5 Monitoreo activo y alertas

- **Wakeup/keep-alive** (notifications/wakeup, interno): despierta los 7 servicios (dispara el cold start, espera 90 s, verifica UP/DOWN/TIMEOUT) — heredado de la operación en Render y útil con scale-to-zero; incluye envío de reporte por Gmail.

- **Reintentos observables:** `outbox_events.status/retryCount/lastError` y `notification_deliveries` son en sí mismos paneles de salud consultables.
- **Alertas de plataforma** (Azure Monitor alerts, action groups): no se encontró configuración en el repositorio. *Información pendiente de confirmar con el equipo de desarrollo* (posiblemente en el repo de Terraform).

22.6 Tracing distribuido

El tracing se apoya en Application Insights (correlación automática de operaciones del SDK) más el `correlationId` de negocio en headers y sobres de eventos. No hay OpenTelemetry explícito ni instrumentación manual de spans en el código revisado — la trazabilidad cross-servicio por eventos depende del campo `correlationId` (hoy null en casos donde el caso de uso no lo propaga, según el catálogo de orders).

23. Diagrama de despliegue

[Insertar aquí el diagrama]

Explicación del diagrama

El despliegue reparte el sistema en cuatro dominios de infraestructura. En **Azure**, dentro del grupo `eciexpress-prod-rg`, viven el frontend (Static Web Apps, con la configuración embebida en el bundle) y las nueve Container Apps (`eciexpress-prod-<servicio>`, nombres verificados en los workflows): solo el gateway tiene ingress externo — el resto se alcanza por FQDN internos del entorno, materializando el perímetro único. Las imágenes provienen del ACR con tag `production`, y cada deploy crea una **revisión** con sufijo `<sha>-<run>` (trazabilidad binario↔commit; el workflow documenta por qué el sufijo es necesario). El plano de datos Azure completa el cuadro: Service Bus con Managed Identity (sin secretos), Redis con TLS, Blob para todos los binarios y Application Insights para la telemetría.

Las **bases de datos no están en Azure**: Neon provee los seis PostgreSQL serverless con TLS obligatorio — decisión de costo/simplicidad coherente con el contexto académico. La **GPU tampoco**: la API Hunyuan3D corre en una instancia EC2 con NVIDIA A10G en AWS, consumida por `products-service` a través de una URL configurable — el diagrama la muestra como la única pieza de cómputo fuera de Azure. Los SaaS externos (Firebase, Wompi, Gmail, Meta, Twilio) cierran el perímetro; nótese la flecha entrante de Wompi hacia el gateway: el único tráfico externo que no proviene de usuarios.

GitHub es el plano de control de despliegue: pipelines por repositorio con login OIDC (federación de identidad workload↔Azure), build y tests como guardas, y limpieza de manifests sin tag en ACR como mantenimiento del registry.

24. Ambiente de desarrollo y producción

24.1 Ambientes identificados

Ambiente	Propósito	Evidencia
Desarrollo individual	Un servicio con npm/pnpm run start:dev, .env local, Swagger auto-abierto, Redis opcional (degrada a memoria), mocks disponibles (ProductsMockClient), AUTH_DISABLED posible	main.ts de orders (auto-open Swagger en NODE_ENV=development), .env.example por servicio
Integración local (Docker Compose)	Los 7 servicios + gateway + Redis efímero + emulador de Azure Service Bus (con SQL Edge), red interna eci, solo el gateway publicado	docker-compose.yml (encabezado: "stack local para PRUEBAS DE INTEGRACIÓN"), servicebus-config.json
Producción (Azure)	Container Apps + SWA + Service Bus real (Managed Identity) + Redis Azure + Neon + App Insights	Workflows, .env.example del gateway (URLs internas ACA, rediss://)

No se encontró un ambiente de *staging* separado. El job de deploy usa el environment production de GitHub (que permite reglas de aprobación). *Información pendiente de confirmar con el equipo de desarrollo.*

24.2 Diferencias clave entre ambientes

Dimensión	Desarrollo	Integración local	Producción
Mensajería	Emulador o RabbitMQ legado según .env	Emulador Service Bus (connection string UseDevelopmentEmulator=true)	Service Bus real, Managed Identity + FQNS
Base de datos	Neon compartida de dev o Postgres local	Neon (la nube se reutiliza: "Postgres (Neon)... NO se levantan aquí")	Neon producción, TLS
Redis	Opcional (memoria)	Contenedor redis:7-alpine efímero sin persistencia	Azure Cache for Redis (TLS)
Auth	AUTH_DISABLED posible en orders; tokens reales recomendados	Tokens reales; orders en modo header detrás del gateway	Firebase + sesión + enriquecimiento completos

Exposición	Cada servicio en su puerto local	Solo gateway :3000 publicado	Solo gateway con ingress externo
Config	.env + carga manual (loadLocalEnv)	env_file + overrides del compose	Secrets/env de Container Apps; VITE_* en build del SWA
Observabilidad	App Insights no-op sin connection string; logs consola	Ídem	App Insights activo
Wompi	Sandbox	Sandbox	Sandbox (URL del webhook debe apuntar al gateway desplegado)

24.3 Pipeline de entrega (backend)

```

push a main → build (pnpm install --frozen-lockfile + pnpm build)
              → test (pnpm test) [gate obligatorio]
              → deploy (solo main, environment production):
                  az login OIDC → docker build → push ACR :production
                  → az containerapp update --revision-suffix <sha7>--<run#>
                  → limpieza de manifests sin tag (>3 días, best effort)

```

Frontend: build (npm ci + lint informativo + vite build) → deploy a SWA con Azure/static-web-apps-deploy@v1 (PRs también compilan; solo main publica).
Concurrency por rama con cancelación de ejecuciones viejas.

25. Stack tecnológico

25.1 Frontend

Categoría	Tecnología	Uso
Framework	React 19 + TypeScript	SPA
Build	Vite (tsc -b && vite build)	Bundling, VITE_* en build time
Estilos	Tailwind CSS 4 (+ clsx, tailwind-merge)	Sistema de estilos utilitario
Routing	react-router-dom	Navegación por páginas
Estado/red	fetch propio tipado (services/api.ts + servicios por dominio), context + hooks, pageCache	Acceso HTTP con Bearer + X-Session-Id

Tiempo real	socket.io-client	Chat y campana
Auth	firebase (SDK web)	Login Google/email
3D	@google/model-viewer	Visor GLB de productos
Mapa	maplibre-gl	Mapa del campus/edificios
QR	qr-scanner	Escaneo del código por el vendedor
UX	framer-motion, lucide-react, react-toastify, react-responsive	Animación, iconos, toasts, breakpoints

25.2 Backend

Categoría	Tecnología	Servicios
Runtime / Framework	Node.js 20/22, NestJS 11, TypeScript estricto	Todos
ORM / Data	Prisma 7 (+@prisma/adapters)	identity
	TypeORM + pg	products, orders, financial, fulfillment, notifications
	pg (Pool directo, read-only)	reporting
Validación	Zod (identity); class-validator/class-transformer (resto)	—
Documentación	@nestjs/swagger + swagger-ui-express	Todos (no expuesto públicamente)
WebSockets	@nestjs/websockets + socket.io	orders, notifications
Jobs	@nestjs/schedule + cron	products (retry 3D), fulfillment (expiración), notifications (reintentos), identity (cierres — worker propio)
Salud	@nestjs/terminus	products (referencia); /health manual en el resto
PDF / Imagen / QR	pdfkit (financial); sharp + qrcode (fulfillment); multer (products)	—
HTTP client	axios / @nestjs/axios	orders, products, financial, notifications, reporting
Logging	pino / nestjs-pino / winston / StructuredLogger propio	según servicio (\$22.2)

25.3 Mensajería

Elemento	Tecnología
Broker	Azure Service Bus (@azure/service-bus), topic eciexpress_events, suscripción por servicio, subject = routing key
Autenticación	Managed Identity (@azure/identity, DefaultAzureCredential) en prod; connection string + emulador local
Patrones	Transactional Outbox (identity/orders/fulfillment), consumidores idempotentes, DLQ por MaxDeliveryCount
Legado	RabbitMQ/CloudAMQP (migrado; restos en .env.example y docs — ver DT-01)

25.4 Persistencia

Elemento	Tecnología
Relacional	PostgreSQL serverless en Neon (6 bases, database-per-service, TLS)
Objetos	Azure Blob Storage (products, store-logos/banners, avatares, evidencias de devolución, recibos, modelos GLB)
Caché	Redis 7 (local) / Azure Cache for Redis (prod, TLS); in-memory fallback

25.5 Cloud e infraestructura

Elemento	Tecnología
Cómputo backend	Azure Container Apps (env compartido, ingress interno salvo gateway, revisiones por deploy)
Frontend hosting	Azure Static Web Apps
Registry	Azure Container Registry (tag production, limpieza de untagged)
Telemetría	Azure Application Insights + Azure Monitor (Logs KQL y Metrics)
GPU IA	AWS EC2 (NVIDIA A10G) para Hunyuan3D API (Python: PyTorch/CUDA, Trimesh)
IaC	Terraform (referenciado por outputs/reglas; código no incluido en este repositorio)
Contenedores locales	Docker Compose + emulador Service Bus + Azure SQL Edge + Redis alpine

25.6 CI/CD

Elemento	Tecnología
Orquestador	GitHub Actions (un workflow por repo)
Autenticación cloud	OIDC (id-token) → Service Principal, sin secretos de larga vida
Flujo backend	build → test → docker build → push ACR → az containerapp update --revision-suffix
Flujo frontend	build Vite (+lint informativo) → SWA deploy (PR builds; publica solo main)
Gestión de paquetes	pnpm 9 (backend, --frozen-lockfile), npm ci (frontend)

25.7 Servicios externos (SaaS)

Servicio	Función
Firebase Auth	Identidad federada y tokens
Wompi (sandbox)	Recargas: tarjeta tokenizada, Nequi, PSE, Bancolombia; webhook firmado
Gmail API (OAuth2)	Correo transaccional
Meta WhatsApp Cloud API v25.0	WhatsApp (texto, plantilla, imagen)
Twilio	SMS
GitHub API	Estado de deploys para el centro de monitoreo

25.8 Herramientas de desarrollo

Herramienta	Uso
Jest + Supertest	Unit y e2e por servicio (gate del pipeline)
ESLint 9 + Prettier	Estilo y calidad
Swagger UI	Exploración de APIs en desarrollo (auto-open en orders)
Scripts de prueba de eventos	scripts/publish-test-event.js (orders) para simular eventos entrantes

26. Resumen de decisiones arquitectónicas

#	Decisión	Motivo	Ventajas	Consecuencias/Trade-offs
ADR-01	Microservicios por subdominio (7) con repos y pipelines independientes	Equipos paralelos por dominio; evaluación y despliegue autónomos	Autonomía, aislamiento de fallos, escala selectiva	Complejidad operativa; necesidad de contratos formales y stack local de integración
ADR-02	NestJS + TypeScript en todo el backend	Homogeneidad de conocimiento; DI, guards, pipes y WS de fábrica	Curva compartida entre equipos; patrones uniformes	Acoplamiento al ecosistema Node para cargas CPU-bound (la IA se externalizó a Python/GPU)
ADR-03	API Gateway propio en NestJS (no producto de mercado)	Necesidades específicas: raw body Wompi, reescrituras por servicio, enriquecimiento con caché, WS proxy con reescritura de userId	Control total; mismo stack; testeable	Mantenimiento propio; TODO conocido de auth Socket.IO; SPOF a escalar con réplicas
ADR-04	Firebase Auth como IdP; perfiles locales enlazados por firebaseUid	No custodiar credenciales; social login inmediato	Menos superficie de riesgo; MFA/recuperación delegadas	Dependencia de Google; doble fuente usuario reconciliada por sync-profile
ADR-05	Bus de eventos único (topic eciexpress_events) con routing keys servicio.entidad.accion	Coreografía de la saga y fan-out a interesados	Desacoplamiento temporal; extensibilidad; auditabilidad	Razonamiento distribuido; requiere catálogos y monitoreo de flujo de eventos
ADR-06	Migración RabbitMQ/CloudAMQP → Azure Service Bus (subject=routing key, suscripción por servicio, reglas SQL)	Consolidación en Azure; Managed Identity (sin secretos); emulador oficial para local	Menos credenciales; Docs/.env legados DLQ/lock/reintentos desactualizados (DT-gestionados; misma 01); dependencia de semántica topic	Azure
ADR-07	Transactional Outbox en productores críticos (identity, orders, fulfillment)	Publicar y persistir atómicamente; sobrevivir caídas del broker	Cero eventos perdidos; reintentos con backoff observables	Tabla y worker extra por servicio; latencia de publicación (poll 5 s)

ADR-08	Consumidores idempotentes en todos los servicios (processed_events, unicidad de negocio, advisory locks)	Entrega al-menos-una-vez del bus	Reprocesos seguros; carreras resueltas	Disciplina transversal: todo evento requiere idempotencyKey
ADR-09	orders como único dueño del estado del pedido (saga coreografiada, sin orquestador)	Un solo lugar decide transiciones; los demás publican hechos	Máquina de estados auditable; sin SPOF de orquestación	orders concentra complejidad (11 eventos de entrada)
ADR-10	Carrito = orden DRAFT + cotización autoritativa de products (cartId == orderId)	Un solo agregado desde el primer clic; precios/promos de una sola fuente	Sin doble modelo carrito/orden; precios congelados verificables	Ida-y-vuelta asíncrono para cotizar (UX de "cotizando...")
ADR-11	Reserva de stock por eventos con gate de confirmación (reservation_confirmed + stockReserved; "Option C" para efectivo)	Evitar sobreventa de la última unidad	Sin dobles confirmaciones; efectivo espera stock real	Estados intermedios visibles; compensaciones con wasSold
ADR-12	Escrow financiero HELD→RELEASED con desglose fijado en checkout	"Precio visto = precio cobrado"; pagar al negocio solo tras entrega	Confianza del comprador; comisión transparente; reembolsos con política	Requiere evento de entrega confiable (outbox en fulfillment)
ADR-13	Recargas vía Wompi con webhook firmado sobre raw body	No custodiar datos de tarjeta; confirmación asíncrona del PSP	PCI fuera de alcance; evidencia completa (wompi_response jsonb)	El gateway debe preservar el buffer intacto; URL del webhook por ambiente
ADR-14	Database-per-service en Neon + proyecciones locales + reconciliación pull	Autonomía de datos; costo serverless	Sin acoplamiento por esquema; hot path sin llamadas síncronas	Consistencia eventual; duplicación controlada; reporting necesita excepción read-only
ADR-15	QR como token opaco en BD (no JWT) + código corto base32	Un solo uso, invalidable, regenerable; operable sin cámara	Revocación trivial; imagen pública inadivisible para correos/WhatsApp	Estado adicional; job de expiración necesario
ADR-16	Notificaciones dirigidas por catálogo con canales enchufables y	Publicar el evento de dominio debe bastar	Integración de equipos sin conocer canales; sandbox sin credenciales;	Catálogo a mantener; contratos de payload sensibles (caso userId de financial)

	bandeja siempre persistida		preferencias por usuario	
ADR-17	Container Apps + ACR + OIDC + revisión por commit	PaaS de contenedores con escala; sin secretos de larga vida	Deploys trazables; rollback por revisión; menos gestión de infra	Restricciones de la suscripción (ACR Tasks bloqueado) sorteadas con Docker build
ADR-18	Generación 3D externalizada (Hunyuan3D en GPU AWS) con estado de progreso en el producto	Node no es apto para inferencia GPU; experiencia 3D como diferencial	Pipeline asíncrono con reintentos; catálogo no bloqueado por la IA	Dependencia cross-cloud; timeout 10 min; costo GPU
ADR-19	reporting sin BD propia: KQL + lecturas read-only allowlisted	Evitar duplicar datos para analítica en MVP	Cero sincronización; seguridad por catálogo de consultas	Acoplamiento débil a esquemas de orders/financial (solo lectura)
ADR-20	Sesión única con X-Session-Id (TTL 8 h) además del token Firebase	Control de sesión del lado servidor (revocación, single-session)	Cierre de sesión efectivo server-side	Header adicional a propagar; estado de sesión en identity

27. Riesgos técnicos

#	Riesgo	Impacto	Probabilidad	Mitigación actual / recomendada
R-01	Gateway como punto único de falla (toda la autenticación y el tráfico pasan por él)	Alto: caída total del acceso	Media	Estado ya externalizado a Redis (permite N réplicas); recomendado: mínimo 2 réplicas y autoscaling; health probe de ACA
R-02	Modelo de confianza por headers depende del aislamiento de red — si un servicio interno quedara expuesto, x-user-id sería falsificable (orders opera con AUTH_DISABLED detrás del gateway)	Alto: suplantación de identidad	Baja (requiere error de configuración de ingress)	Bloqueos en gateway + ingress interno ACA; recomendado: verificación periódica de ingress y/o token service-to-service firmado
R-03	Autenticación WebSocket incompleta (TODO(ws-auth))	Medio: fricción de integración	Media	Ya mitigado parcialmente (se elimina ?token= antes del

	documentado): el token debe ir en ?token= del upgrade; clientes que solo usen auth.token son rechazados; el token viaja en URL	y exposición del token en logs		downstream); recomendado: bridge Socket.IO en el gateway
R-04	Pérdida de eventos en productores sin outbox (products y financial publican directo al bus) — si el proceso cae entre el commit de BD y el publish, el evento se pierde	Alto en la saga	Baja-media	El compose llegó a incluir un “reconciler” (hoy desactivado); recomendado: outbox también en products/financial o job reconciliador productivo
R-05	Dead-letter queues sin consumidor visible — mensajes que agoten MaxDeliveryCount quedan en DLQ sin proceso de reproceso documentado	Medio: hechos de negocio varados	Media	reporting expone métricas del bus; recomendado: alarma sobre DLQ y runbook de reproceso
R-06	Socket.IO sin adapter distribuido — con más de una réplica de orders/notifications, los broadcasts a salas se fragmentan por instancia	Medio: mensajes en vivo perdidos entre réplicas	Media si se escala	Documentar límite de 1 réplica o incorporar adapter Redis de Socket.IO
R-07	Dependencias de terceros gratuitos/sandbox (Neon, Wompi sandbox, cuota Gmail, emulador SB local)	Medio: interrupciones o límites de cuota	Media	Multi-proveedor ya abstraído por interfaces (canales); presupuestar tiers pagos para producción real
R-08	GPU única en AWS para el pipeline 3D (instancia EC2; sin evidencia de HA)	Bajo (funcionalidad no crítica para vender): productos sin modelo 3D	Media	Estados modelGenerationStatus/Error + retry-3d + scheduler ya degradan elegantemente; ai-health para monitoreo
R-09	TTL del caché de enriquecimiento (60 s) — ventana en la que un usuario suspendido o con rol cambiado conserva su acceso anterior	Bajo-medio	Alta (por diseño)	Aceptado y documentado; si se requiere revocación inmediata: invalidación activa de la clave gw:enrich:<uid>
R-10	Contratos de eventos gobernados por documentos que ya divergieron (casos reales: RabbitMQ vs Service Bus, Resend vs Gmail, userId faltante en eventos de financial)	Medio: integraciones silenciosamente rotas (notificaciones omitidas)	Alta (ya ocurrió)	Este documento reconcilia el estado; recomendado: contratos como código compartido (paquete npm de tipos) o schema registry

R-11	Nota legal de Wompi: operar dinero real exige regulación SFC; el modelo retiene y desembolsa fondos de terceros	Alto si se pasa a producción real sin cumplimiento	Baja en contexto académico	Nota explícita en wompi.service.ts; permanecer en sandbox hasta asesoría legal
R-12	Credenciales en docker-compose.yml histórico — el bloque comentado del reconciler contiene una cadena de conexión Neon con contraseña	Medio: fuga de credenciales si el repo se hace público	Media	Rotar esa credencial y mover a secrets; auditar historial de Git

28. Deuda técnica

#	Deuda	Evidencia	Prioridad	Recomendación
DT-01	Documentación de mensajería desactualizada tras la migración a Service Bus: READMEs y catálogos siguen diciendo "RabbitMQ (CloudAMQP)", "worker BullMQ", RABBITMQ_URL en .env.example; fulfillment docs dicen "Prisma 7" pero usa TypeORM	identity-service/README.md §4/§9, orders-service/docs/CATALOGO-DE-EVENTOS.md §1, notifications-service/docs/INTEGRACION-EVENTOS.md §1/§4, products-service/.env.example, fulfillment-service/docs/casos-de-uso-mvp.md	Alta (induce a error a nuevos integrantes)	Actualizar los 4 documentos y limpiar variables legadas; enlazar este documento como fuente
DT-02	Guía de notificaciones promete canales retirados/no implementados: Push/FCM fue eliminado por migración (RemovePush) pero la guía documenta POST /devices y canal PUSH;	notifications-service/docs/INTEGRACION-EVENTOS.md §3/§5 vs migrations/1750000000000-RemovePush.ts y email.channel.ts	Media	Reescribir §5 de la guía; retirar referencias a FCM/Resend

	email real es Gmail API, no RESEND_API_KEY			
DT 03	financiamiento no distingue - paymentMethod en order.order.created: el catálogo de orders afirma que financiero "lo omite si paymentMethod === 'cash'", pero el payload de financiero no declara paymentMethod (TODO explícito) y handleOrderCreated debita la billetera siempre → un pedido en efectivo podría generar débito o payment.failed espurio	financiamiento- service/src/events/payloads/order.payl oads.ts (TODO), transactions.service.ts#handleOrderCre ated	Alta (riesgo funcional en el flujo efectivo)	Añadir paymentMethod al payload consumido y saltar el débito para cash; test de integración del flujo efectivo
DT 04	Eventos de financiero sin userId del comprador: notifications omite notificaciones de topup.approved, payment.processed/failed, refund.issued si no llega userId (advertencia vigente en la guía)	INTEGRACION-EVENTOS.md §3.4 (A)	Alta (UX silenciosamente degradada)	Incluir userId en los 4 payloads (financiero ya conoce la wallet→user)
DT 05	AdminGuard de financiero es placeholder: el gateway lo anota como "TODO en Financiero" — endpoints /admin/* de financiero dependen solo del header de rol	GATEWAY_RESUMEN.md §3, financiamiento- service/src/common/guards/admin.gu ard.ts	Alta (autorización)	Implementar verificación real de rol/permiso en el guard
DT 06	Outbox ausente en products y financiero (publican directo al bus)	Estructura de src/messaging/src/events sin tablas outbox; contraste con identity/orders/fulfillment	Media-alta	Introducir outbox o reconciliador (ver R-04)

DT 07	Reconciler local roto y desactivado: el compose referencia scripts/event-reconciler.mjs que no existe ("DESACTIVADO (2026-07-08)... MODULE_NOT_FOUND"); la carpeta scripts/ raíz está vacía	docker-compose.yml líneas 168-191	Media	Aportar el script o retirar el bloque; considerar su versión productiva
DT 08	/notifications/wakeup sin mecanismo de auth definido (bloqueado en gateway con TODO); menciona esperas de cold start de Render (legado)	route-config.ts, wakeup.service.ts	Media	Definir secreto/manejo o interno; recalibrar tiempos para ACA
DT 09	PENDING_STOCK existe solo en tipos del frontend — el backend usa CREATED + flag stockReserved	frontend/ecixpress/src/lib/orders-api.ts, orders-ui.ts vs order.models.ts	Baja	Alinear el union type del front con los 13 estados reales
DT 10	Lint del frontend no bloquea el pipeline (continue-on-error: true)	azure-swa-deploy.yml	Media	Corregir errores ESLint y volver el lint gate obligatorio
DT 11	READMEs plantilla en varios repos (financial y frontend conservan el README por defecto de NestJS/Vite)	financial-service/README.md, frontend/ecixpress/README.md	Baja	README mínimo por servicio: rol, puertos, eventos, cómo correr
DT 12	.env con secretos presentes en el árbol de trabajo local y credencial Neon en bloque comentado del compose	presencia de .env en cada servicio (gitignored) y docker-compose.yml	Media	Rotación y verificación de no-commit; secret scanning en CI
DT 13	Ack-siempre en el consumer de orders: los errores de handler no reencolan (evita poison loops) pero	service-bus.service.ts de orders (comentario)	Media	Métrica+alert a sobre errores de handler;

	pueden requerir reproceso manual sin herramienta dedicada			comando de reproceso
DT - 14	Swagger auto-open con ejecución de comandos de sistema en main.ts de orders (detección de navegador vía ps aux/tasklist) — conveniencia de dev dentro del código de producción	orders-service/src/main.ts	Baja	Extraer a script de dev fuera del bootstrap

29. Glosario técnico

Término	Definición en el contexto ECIEXPRESS
ACA (Azure Container Apps)	PaaS de contenedores donde corren gateway y microservicios; cada deploy crea una <i>revisión</i>
ACR (Azure Container Registry)	Registro privado de imágenes Docker (<acr>.azurecr.io/<servicio>:production)
ACME / HTTP-01	Protocolo de emisión de certificados TLS; el gateway responde el challenge en /.well-known/acme-challenge
Advisory lock	Bloqueo aplicativo de PostgreSQL (pg_advisory_xact_lock) usado por fulfillment para serializar eventos duplicados concurrentes
Application Insights	Servicio de telemetría de Azure; recibe requests, dependencias, excepciones y trazas de los 8 servicios
Binding / Routing key	Patrón de enrutamiento de eventos servicio.entidad.accion (p. ej. order.order.created); en Service Bus viaja como subject
Carrito (DRAFT)	Orden en estado borrador que acumula ítems; cartId == orderId por invariante
Checkout	Transición del carrito a pago/confirmación; fija el recargo de hora pico y publica order.order.created
Consistencia eventual	Régimen del sistema entre servicios: los hechos se propagan por eventos y las réplicas convergen
Coreografía (saga)	Coordinación distribuida sin orquestador: cada servicio reacciona a eventos y publica los suyos

Correlation ID	Identificador de traza extremo a extremo; nace como x-correlation-id en el gateway y viaja en el sobre de eventos
DLQ (Dead-Letter Queue)	Subcola de Service Bus donde caen mensajes tras exceder MaxDeliveryCount (10)
Enriquecimiento	Traducción firebaseUid → {userId, roles, storeId, status} que hace el gateway contra identity (caché 60 s)
Envelope (sobre)	Metadatos estándar de todo evento: eventVersion, source, correlationId, occurredAt, idempotencyKey
Escrow	Retención del cobro (HELD) hasta la entrega; liberación (RELEASED) acredita el neto al negocio
Firebase ID token	JWT firmado por Google que prueba la identidad del usuario; se verifica con el Admin SDK
FQNS	Fully Qualified Namespace de Service Bus (<ns>.servicebus.windows.net), usado con Managed Identity
GLB	Formato binario de modelo 3D generado por Hunyuan3D y renderizado con model-viewer
Hora pico (peak fee)	Recargo porcentual por franja de alta demanda configurado por tienda en financiero y fijado en el checkout
Hunyuan3D	Modelo de Tencent para generar 3D desde imágenes; expuesto como API Python propia en GPU
Idempotency key	UUID que garantiza que un evento/req se procese una sola vez aunque se entregue varias veces
Managed Identity	Identidad administrada de Azure usada por los servicios para autenticarse en Service Bus sin secretos
Neon	PostgreSQL serverless gestionado donde viven las 6 bases de datos
OIDC (CI/CD)	Federación GitHub↔Azure que evita guardar credenciales cloud en GitHub
Option C (efectivo)	Regla de confirmación de pedidos en efectivo: CREATED → espera reservation_confirmed → CONFIRMED
Outbox transaccional	Patrón que persiste el evento junto al cambio de negocio y lo publica un worker con reintentos
PQRS	Peticiones, Quejas, Reclamos y Sugerencias — módulo de tickets de soporte en identity
Proyección	Réplica local de solo lectura de datos de otro servicio, alimentada por eventos (a veces con reconciliación pull)
PSE / Nequi / Bancolombia	Métodos de pago colombianos soportados en las recargas vía Wompi
QR de retiro / short code	Credencial de un solo uso para retirar el pedido: token opaco (QR) y código corto base32 tecleable
Rate limiting	Límite de 60 req/60 s por usuario/IP en el gateway, con conteo distribuido en Redis
RBAC	Control de acceso por roles (BUYER, VENDOR, ADMIN, ANALYST) + permisos recurso:acción

Reconciliación pull	Ante un evento-notificación, el consumidor consulta el snapshot completo al servicio dueño
Reserva de stock	Compromiso de unidades (reserved_stock) previo a la confirmación; se consume, libera o restituye
Revisión (ACA)	Versión inmutable desplegada de un Container App; sufijo <sha7>-<run#> por deploy
Saga	Transacción distribuida como secuencia de pasos compensables coordinados por eventos
Service Bus (topic/suscripción)	Broker pub/sub de Azure: un topic compartido y una suscripción filtrada por consumidor
Sesión única (X-Session-Id)	Sesión server-side de 8 h emitida por identity; complementa al token de Firebase
SPA	Single-Page Application (React) servida por Static Web Apps
SWA	Azure Static Web Apps, hosting del frontend
wasSold	Flag de order.order.cancelled que indica si el stock ya se había descontado (restituir) o solo reservado (liberar)
Wompi	Pasarela de pagos colombiana usada para recargas; notifica por webhook firmado
Webhook	Llamada HTTP entrante de un tercero (Wompi) hacia el gateway; verificada por firma, no por JWT
WSS	WebSocket sobre TLS; transporte de chat y notificaciones en vivo